TestSplit Technical Guide

Authors: Samson Oloruntola (22714745) & Marjia Siddik (22306501)
Supervisor: Dr Paul Clarke
Module: CSC1097 - Final Year Project
Submission Date: 01/05/2026

Table of Contents

1. Motivation

1.1 Problem Statement

CI has become essential in modern software development, yet slow pipelines remain an obstacle to developer productivity. As documented in the academic and industry literature [1], [2], [3], the root cause is rarely a single test but rather a combination of structural factors that compound over the lifetime of a project.

Four interrelated bottlenecks characterise the problem:

1. **Large test suites.** Projects with hundreds or thousands of unit tests accumulate total execution times that far exceed acceptable feedback windows. What begins as a thirty-second test run can, without intentional management, grow into a ten-minute-or-longer pipeline stage.
2. **Uneven test distribution.** When tests are split across parallel jobs arbitrarily, for example, by file order or alphabetical grouping, some jobs finish in seconds while others run for minutes. The slowest job determines the total pipeline duration, making uneven distribution one of the highest-impact inefficiencies to resolve.
3. **Sequential execution bottlenecks.** Many CI pipelines run their entire test suite within a single job, executing tests one after another. This approach fails to exploit the parallelism available on modern CI runners and leaves significant performance headroom unrealised.
4. **Long developer feedback cycles.** Slow pipelines increase the time between a developer committing code and receiving test results. In agile and continuous-delivery environments, this delay reduces throughput, increases context-switching, and lengthens the overall development cycle [1].

TestSplit addresses these bottlenecks by analysing measured test execution times and applying the Longest Processing Time (LPT) scheduling algorithm [4], [5], [6] to distribute tests across parallel CI jobs in a provably near-optimal fashion. The result is a shorter critical path, a more balanced workload, and a faster feedback loop, without requiring any modification to the tests themselves or the underlying repository structure.

1.2 Objectives

TestSplit is designed around four primary technical objectives that together define the scope of the implementation:

1. **Reduce CI pipeline runtime.** By distributing tests intelligently across parallel jobs using the LPT algorithm, TestSplit aims to reduce the total elapsed time of the test stage in CI pipelines. The system records both a baseline and an optimised run and computes a speed-up factor to make this improvement measurable and verifiable.
2. **Improve workload balance across jobs.** The LPT scheduler minimises the difference in accumulated duration between the fastest and slowest parallel jobs. Post-scheduling metrics, including balanceRatio, idealTime, and criticalPath, are computed and exposed to enable users to assess the distribution's quality quantitatively.

3. **Require minimal changes to existing workflows.** TestSplit integrates with existing CI environments by consuming standard JUnit XML output files and generating ready-to-use YAML configuration files for GitHub Actions [7] and GitLab CI [8]. No modifications to test code, build scripts, or repository structure are required.
4. **Provide historical performance insights.** TestSplit persists profiling data locally across runs and provides a local web dashboard for visualising performance trends, identifying regressions, and comparing baseline against optimised configurations. This enables teams to track improvements over time and detect newly introduced bottlenecks before they become critical.

These objectives are directly reflected in the functional requirements described in the Functional Specification: FR3 (parallelisation and test grouping), FR4 (CI configuration generation), FR9 (historical trend analysis), and FR7 (local dashboard for historical metrics).

1.3 Glossary

Term	**Definition**
Abstract Syntax Tree	A tree representation of the syntactic structure of source code, where each node represents a construct in the source language.
Amdahl’s Law	Amdahl's Law calculates the performance impact resulting from optimizing one or more components of a program.
Balance Ratio	The ratio of minimum to maximum job load across parallel jobs.
Bin Packing	Combinatorial optimisation problem, described as the placement of a set of different-sized items into identical bins such that the number of containers used is optimally minimized.
Coefficient Of Variation (CV)	The coefficient of variation is a statistical measure that quantifies the degree of variability in a set of data by calculating the ratio of the standard deviation to the mean.
Context Switching	where the operating system scheduler switches from executing one thread to another, incurring a delay known as context switch overhead.
Core Affinity Pinning	The binding of a process or thread to a specific CPU core to reduce context switching overhead and improve execution consistency.
Critical Path	The longest sequence of dependent tasks in a schedule, determining the minimum possible duration of the overall process.
Dependency Graph	A directed graph representing dependencies between entities, where an edge from node A to node B indicates that A must be completed before B.
Docker Layer Caching	A build optimisation in which unchanged image layers are reused from a previous build, reducing build time.

Exponential Smoothing	A time series forecasting method that assigns exponentially decreasing weights to past observations, with more recent values weighted more heavily.
First Fit Decreasing (FFD)	A bin packing heuristic that sorts items in descending order of size and places each item into the first bin with sufficient remaining capacity.
Forward-Compatible Migration	A design approach in which data formats or schemas are structured so that future versions of a system can read data produced by earlier versions.
Heuristic	A technique designed to solve a problem more quickly or with less information than exact methods, at the cost of optimality guarantees.
JUnit XML	A widely adopted XML report format for test results, originally defined by the Apache Ant JUnit task and produced by Maven Surefire.
Kahn’s Algorithm	A breadth-first algorithm for topological sorting that detects cycles in a directed graph while producing a valid linear ordering of nodes.
Lifecycle Services	External services such as databases or message brokers required by a test suite at runtime, provisioned as Docker containers within the CI environment.
List Scheduling	A class of scheduling algorithms that process tasks from an ordered list, assigning each task to a machine according to a priority rule.
Longest Processing Time (LPT)	A list scheduling algorithm that assigns tasks in descending order of processing time, each to the machine with the current minimum load, producing a makespan within 4/3 of optimal.
Makespan	The completion time of the last job in a schedule, equal to the maximum of all job completion times.
Maven Wrapper	A script bundled with a Maven project that downloads and invokes a specified Maven version without requiring a system-level installation.
MULTIFIT	A scheduling algorithm that applies FFD iteratively within a binary search to find a makespan close to optimal.
Recency Bias	The tendency to assign greater weight to recent observations than to older ones when estimating a quantity over time.
Regression	A measurable degradation in a system performance metric between successive measurements, indicating a performance defect introduced by a change.
Risk Factor	A configurable multiplier applied to a test's standard deviation during scheduling to increase its effective weight and account for timing variability.

Smoothing Factor	The parameter controlling the rate of decay of older observations in exponential smoothing, where higher values give greater weight to recent data.
Speed-up Factor	The ratio of the time taken to complete a task using a single processor to the time taken using multiple processors in parallel.
Surefire	A Maven plugin that executes unit tests during the build lifecycle and generates JUnit XML test reports.
Test Sharding	The practice of partitioning a test suite into disjoint subsets for concurrent execution across multiple runners to reduce total execution time.
Testcontainers	A Java library that provides lightweight, disposable Docker container instances for use in automated tests.
Topological Sort	A linear ordering of the vertices of a directed acyclic graph such that for every directed edge (u, v), vertex u appears before v.
Unstable Test	A test whose coefficient of variation across recent runs exceeds a defined threshold, indicating non-deterministic or environment-sensitive execution behaviour.
Worker	A process or thread responsible for executing an assigned subset of tasks concurrently with other workers in a parallel execution environment.
Work Stealing	A parallel scheduling strategy in which idle workers dynamically acquire tasks from the queues of busy workers to improve resource utilisation.

Table 1: Glossary of Terms

1.4 Target Users

TestSplit is intended for technically proficient users who work with automated testing and CI/CD infrastructure. Three primary user groups have been identified:

1. **Software development teams using CI/CD.** Any team, from a small startup to a large enterprise, that relies on automated testing as part of a continuous integration workflow stands to benefit from TestSplit. This includes developers, DevOps engineers, QA engineers, and technical leads who are responsible for maintaining pipeline health and shortening feedback cycles. Users at different experience levels are accommodated: beginner users benefit from simple CLI commands and visual dashboards, while advanced users gain access to detailed distribution metrics and configuration control.
2. **Projects with large automated test suites.** TestSplit delivers the most value when the test suite is large enough that sequential execution creates a meaningful bottleneck. The tool is specifically designed for independent unit tests that can be freely reordered and parallelised; test suites with ordering dependencies or multi-step integration workflows are explicitly outside the scope.

Open-source projects such as apache/commons-lang and jhy/jsoup represent the scale and structure of repositories for which TestSplit is intended.

3. **Organisations using GitHub Actions or GitLab CI.** TestSplit generates configuration files specifically for GitHub Actions [7] and GitLab CI [8], the two most widely used CI/CD platforms in open-source and enterprise development. Both platforms support parallel job execution and matrix strategies, which are the mechanisms TestSplit exploits. Support for additional platforms, such as Jenkins and CircleCI, is identified as a potential future enhancement but is outside the scope of the initial implementation.

2. Research

2.1 CI Pipeline Optimisation Techniques

2.1.1 Parallel Test Execution

The most direct response to a slow test suite is to run multiple tests simultaneously. Modern CI platforms such as GitHub Actions [7] and GitLab CI [8] natively support job matrices, in which multiple runner instances execute concurrently within a single pipeline stage. Rather than one runner processing every test in sequence, the workload is divided, and each portion is handled independently, with the stage completing only when the slowest job finishes.

The theoretical upper bound on the improvement achievable through parallelisation is described by Amdahl's Law [9], which states that the speed-up of a workload is limited by its sequential fraction. For a test suite composed entirely of independent unit tests, each executable without dependency on the output of another, the sequential fraction approaches zero, and the potential speed-up grows proportionally with the number of parallel runners. In practice, overhead from job startup, dependency installation, and result aggregation introduces a floor below which additional runners yield diminishing returns [9], [10].

Parallel execution, therefore, resolves the sequential bottleneck identified in Section 1.1, but introduces a new problem: how to divide the work. A naive division, such as alphabetising tests or grouping them by source file, typically produces highly uneven job durations. Because the pipeline stage cannot complete until all parallel jobs have finished, a single overloaded job negates the benefit of parallelising the rest [1].

2.1.2 Scheduling Strategies

The problem of assigning tasks to parallel workers so as to minimise total completion time is a standard topic in scheduling theory, studied extensively since the 1960s [11]. In formal terms, it is an instance of the makespan minimisation problem on identical parallel machines: given a set of jobs with known processing times, assign each job to exactly one machine such that the time at which the last machine finishes, the makespan, is minimised [11].

Several strategies have been proposed and analysed. The simplest is the round-robin assignment, which distributes jobs to machines in a fixed cyclic order regardless of duration. While trivially easy to

implement, round-robin produces poor balance whenever job durations are non-uniform, which is almost always the case in real test suites [12].

List scheduling algorithms improve on this by making assignment decisions based on job durations. The Shortest Processing Time (SPT) rule assigns the shortest available job to the next free machine, minimising average completion time but performing poorly on makespan [12]. The Longest Processing Time (LPT) rule, introduced by Graham [4], takes the opposite approach: jobs are sorted in descending order of duration, and each is assigned to the machine with the least accumulated load. LPT is guaranteed to produce a makespan no worse than 4/3 of the optimal, a bound that tightens as the number of jobs grows relative to the number of machines [4], [5], [6]. For CI test distribution, where the number of tests typically far exceeds the number of parallel jobs, LPT consistently produces near-optimal results in practice.

More complex exact methods, such as integer linear programming and branch-and-bound, can compute provably optimal assignments but require exponential time in the worst case and are impractical for test suites of any meaningful size [11]. Heuristic approaches, including genetic algorithms and simulated annealing, have been proposed for related scheduling problems but carry significant implementation complexity and tuning overhead, making them unsuitable for a tool that must operate quickly and transparently as part of a developer workflow [12].

2.1.3 Test Sharding Approaches

Test sharding is the practice of partitioning a test suite into disjoint subsets, shards, and executing each shard on a separate runner. It is the mechanism by which parallel execution is made concrete in CI environments. The key question is how the partition is formed.

Static sharding assigns tests to shards at configuration time without reference to execution history. A common approach is index-based sharding, in which each test is assigned to a shard (index mod n) where n is the total number of shards. This is simple to configure and requires no historical data, but it produces uneven shard counts when tests have non-uniform durations, which is typical in any project of meaningful complexity [1], [3].

Dynamic or history-based sharding uses measured execution times from previous runs to form a partition whose shards have approximately equal total duration. This approach requires a mechanism for recording and retrieving per-test timings across pipeline runs, as well as an algorithm for computing a balanced partition from those timings. When historical data is available and reasonably stable, history-based sharding consistently outperforms static approaches because it directly targets the source of imbalance: variation in individual test durations [13].

TestSplit implements history-based sharding. Test execution times are parsed from JUnit XML output [14], stored locally across runs, and used as input to the LPT scheduler to produce balanced job groups. This places TestSplit in the category of data-driven CI optimisation tools, as opposed to configuration-driven tools that operate without knowledge of actual test performance.

2.2 Scheduling Algorithms

2.2.1 How LPT Works

The Longest Processing Time (LPT) algorithm is a greedy list-scheduling heuristic for the makespan minimisation problem on identical parallel machines [11], [4]. Its operation is straightforward. Given a set of n tests with measured execution times and a target number of parallel jobs k, the algorithm first sorts all tests in descending order of duration. It then iterates through the sorted list, assigning each test to whichever job currently has the smallest accumulated load. This greedy assignment is typically implemented using a min-heap to keep the per-job load totals efficiently ordered, giving the algorithm an overall time complexity of O(n log k) [11].

The intuition behind sorting in descending order is that large tests, if left to the end, are likely to cause one job to run significantly longer than the others, an imbalance that cannot be corrected once the remaining tests are small. By scheduling the longest tests first, LPT ensures that any remaining imbalance consists only of short tests, keeping the difference between the fastest and slowest jobs small [4], [5].

A concrete example illustrates the point. Suppose five tests have durations of 10, 8, 6, 4, and 2 seconds, and two parallel jobs are available. LPT assigns the 10-second test to Job 1, the 8-second test to Job 2, the 6-second test to Job 2 (now 14 seconds total), the 4-second test to Job 1 (now 14 seconds), and the 2-second test to either job (now 16 seconds versus 14 seconds). The makespan is 16 seconds. A naive split by test order, tests 1, 2, 3 to Job 1 (24 seconds) and tests 4, 5 to Job 2 (6 seconds), yields a makespan of 24 seconds. LPT reduces the critical path by 33% in this case without any additional information beyond test durations [4].

2.2.2 Why It Balances Workloads

LPT balances workloads because its greedy rule directly targets the job most at risk of becoming the bottleneck. At every assignment step, the test is sent to the job with the least accumulated load, which is precisely the job that currently poses the least risk of extending the makespan. This locally optimal choice produces a globally near-optimal result because, by the time all tests are assigned, no single job can be significantly longer than the others without the algorithm having already counteracted that tendency [4].

The theoretical basis for this balance is the 4/3 approximation guarantee proved by Graham [4]: the makespan produced by LPT is at most 4/3 times the optimal makespan for any input [5]. Xiao provided a direct and simplified proof of this bound, and Della Croce and Scatamacchia [6]extended the analysis to tighter bounds for specific input structures. In practice, the 4/3 bound is rarely approached; empirical studies consistently show LPT performing within a few percent of optimal when the number of tests is large relative to the number of jobs, which is the typical case in CI pipelines [12], [6].

Amdahl’s Law [9] provides a complementary perspective: even perfect workload balance cannot eliminate overhead from job startup and result aggregation, so the marginal benefit of additional parallel jobs diminishes beyond a certain point. LPT addresses the controllable component of this ceiling, the imbalance between jobs, while acknowledging that infrastructure overhead places a floor on achievable speed-up regardless of the scheduling quality [9], [10].

2.2.3 Complexity

The time complexity of LPT is O(n log n) when using a comparison sort to order the tests by descending duration, and O(n log k) for the assignment phase when a min-heap is used to track per-job totals, where n is the number of tests and k is the number of parallel jobs [11]. Since $k \leq n$ in all practical scenarios, the sort dominates, and the overall complexity is O(n log n). For a typical CI test suite of several hundred to a few thousand tests, this computation takes milliseconds on any modern machine, making LPT entirely suitable for use in an interactive developer tool [11], [4].

Space complexity is O(n + k): O(n) to store the sorted test list and O(k) for the min-heap of job totals. This is negligible in practice. By contrast, exact makespan optimisation is NP-hard in the general case [11], meaning that any algorithm guaranteed to find the true optimum requires time exponential in the input size. For a test suite of even modest size, exact methods are computationally infeasible, further reinforcing LPT as the appropriate choice [11], [12].

2.2.4 Suitability for CI Pipelines

LPT is well-suited to the CI test distribution problem for several reasons that align directly with TestSplit's constraints and goals.

First, CI test suites are composed of independent unit tests [4]. The LPT algorithm assumes that all tasks can be executed in any order and on any machine without dependency constraints, an assumption that holds exactly for independent unit tests and that TestSplit explicitly enforces as a precondition of use.

Second, historical execution times are available. LPT requires known job durations as input; without them, it degenerates to an uninformed assignment. TestSplit provides these durations by parsing JUnit XML output from previous runs [14] and storing them locally. This means LPT can be applied with accurate, project-specific timing data rather than estimates.

Third, the algorithm is transparent and deterministic. For a developer or DevOps engineer reviewing a generated CI configuration, it is possible to trace exactly why each test was assigned to each job: the sort order and assignment rule are both simple and easy to inspect. This is a meaningful practical advantage over black-box optimisation methods, where the rationale for a given assignment may be opaque [12].

Fourth, LPT requires no tuning parameters. Algorithms such as simulated annealing or genetic algorithms require careful calibration of temperature schedules, mutation rates, or population sizes. LPT has a single input, the number of parallel jobs k, which maps directly to a CI configuration option that developers already understand and control [12].

Taken together, these properties make LPT the most appropriate scheduling algorithm for TestSplit’s use case: it is fast, theoretically grounded, transparent, and directly applicable to the structure of CI test distribution without modification or approximation.

2.3 Existing Solutions and Comparative Analysis

Several approaches exist for reducing CI pipeline duration through test parallelisation, ranging from entirely manual processes to dedicated third-party tools. This section evaluates each approach against the core requirements identified in Section 1.2: reduced runtime, balanced workload, minimal workflow changes, and historical performance insight, and establishes where TestSplit sits relative to existing options.

2.3.1 Manually Splitting

The most basic approach is for developers to manually partition the test suite, assigning specific test files or packages to separate CI jobs in the pipeline configuration. This requires no tooling and can be applied to any CI platform, but it has significant drawbacks in practice [15].

Manual splits are based on developer intuition rather than measured execution times, so they rarely produce balanced job groups. A developer may assign an equal number of test files to each job without knowing that one file contains a single ten-second test while another contains fifty sub-millisecond unit tests. The resulting imbalance is invisible until the pipeline runs, and correcting it requires another manual edit [1], [3].

More critically, manual splits do not self-correct as the codebase evolves. Every time tests are added, removed, or refactored, the split becomes progressively less accurate and must be maintained by hand. In active codebases, this is an ongoing maintenance burden that teams frequently deprioritise, allowing imbalance to accumulate over time [15].

2.3.2 Built-in CI Parallelism

Both GitHub Actions [7] and GitLab CI [8] provide native mechanisms for running jobs in parallel. GitHub Actions supports a matrix strategy that spawns multiple job instances with different parameters, and GitLab CI supports parallel job definitions with a configurable number of jobs. These features are easy to enable and require no external tooling, making them the most common starting point for teams introducing parallelism into their pipelines.

However, neither platform includes any mechanism for distributing tests based on execution time. The developer remains responsible for deciding how to partition the test suite across jobs, which reduces the problem to manual splitting with slightly better configuration ergonomics. Index-based sharding, assigning each test to a shard using a modular index, is a common workaround, but as noted in Section 2.1, it produces uneven distributions whenever test durations are non-uniform [1], [2].

Neither platform provides historical timing data or any feedback mechanism to help developers identify or correct imbalanced distributions. The CI configuration reflects what the developer specified at setup time, not what the pipeline has learned from running [7], [8].

2.3.3 Knapsack Pro

Knapsack Pro is a commercial test parallelisation service that addresses the distribution problem more directly than built-in CI tooling [16]. It operates by maintaining a cloud-hosted queue of test files, sorted

by execution time recorded from previous CI runs. Parallel CI nodes connect to the Knapsack Pro API at runtime and consume tests from the queue in batches, with each node continuously pulling new work until the queue is exhausted. This dynamic Queue Mode means that the distribution adapts to actual node performance during each build, rather than being fixed at configuration time [16].

The tool supports a wide range of test runners and CI platforms, and its cloud-based approach means it can handle flaky tests and variable node boot times without producing idle runners [16]. For teams already using paid CI infrastructure, Knapsack Pro can deliver a near-optimal balance with relatively low setup effort.

However, Knapsack Pro's architecture introduces dependencies that are inappropriate for certain use cases. Test execution metadata is transmitted to and stored on Knapsack Pro's servers, requiring an API token and a network connection at every pipeline run [16]. This creates a runtime dependency on an external service: if the Knapsack Pro API is unavailable, pipelines must fall back to a static, unbalanced split. It also means test metadata, including test names and durations, leaves the team's own infrastructure, which is problematic for projects with data residency requirements or strict privacy constraints [3]. The service operates on a subscription model, which places it out of reach for open-source projects and small teams without tooling budgets.

2.3.4 BuildPulse

BuildPulse is a CI observability platform that focuses primarily on flaky test detection rather than on parallelisation. It works by ingesting JUnit XML test reports uploaded from CI runs and comparing pass/fail outcomes for the same test across commits with the same git tree SHA. When a test is observed to pass and fail on identical code, BuildPulse flags it as flaky and surfaces it in a centralised dashboard with metrics on disruption frequency, failure history, and root-cause context [17].

BuildPulse also offers quarantining, automatically excluding known-flaky tests from blocking CI builds, and integrations with issue trackers such as Jira and GitHub Issues to route detected flakiness into engineering workflows [17]. These capabilities address a different dimension of pipeline reliability than test distribution: rather than reducing total runtime through parallelisation, BuildPulse aims to improve build stability by surfacing and managing non-deterministic test behaviour.

Like Knapsack Pro, BuildPulse operates as a cloud service. Test results are uploaded to BuildPulse's servers for analysis, requiring account registration, API credentials, and ongoing network access [17]. It does not generate CI configuration files or provide any mechanism for distributing tests across parallel jobs, so it does not substitute for a test-splitting tool. Rather, it complements one. For the purposes of this comparison, BuildPulse addresses test reliability but not test distribution.

2.3.5 Why TestSplit Is Useful

TestSplit occupies a position that none of the above approaches fills: a local, open-source, data-driven test distribution tool that operates entirely without external dependencies, cloud infrastructure, or ongoing cost.

Unlike manual splitting and built-in CI parallelism, TestSplit uses measured execution times, parsed from JUnit XML output [14], rather than developer estimates or arbitrary indices. The LPT algorithm [4]then

produces a partition that is theoretically near-optimal and quantifiably better than uninformed approaches, with post-scheduling metrics such as balanceRatio and criticalPath exposed, allowing the quality of the distribution to be assessed directly.

Unlike Knapsack Pro, TestSplit stores all profiling data locally. No test metadata is transmitted to external servers, no API key is required, and no subscription is needed. There is no runtime dependency on a third-party service, meaning the tool works identically whether the developer has network access or not. This makes it suitable for open-source projects, academic contexts, and organisations with data residency constraints [3].

Unlike BuildPulse, TestSplit directly addresses test distribution across parallel jobs. It generates ready-to-use YAML configuration files for GitHub Actions [7] and GitLab CI [8] that developers commit to their repositories and run like any other CI configuration. No agents, webhooks, or platform accounts are required [15].

Finally, TestSplit provides visibility into historical trends through a local dashboard, allowing teams to track speed-up improvements, detect distribution regressions, and validate that the split remains balanced as the test suite evolves. This closes a feedback loop that neither manual splitting nor built-in CI parallelism, nor simpler static tools, provide [1], [2], [3].

Approach	**Distribution method**	**Uses historical data**	**Setup effort**	**Imbalance risk**
Manual splitting	Developer judgement	No	High (ongoing)	High
CI index sharding	Modular index	No	Low	High
Built-in CI parallelism	Platform default	No	Low	Medium
Knapsack Pro	Dynamic queue (cloud)	Yes	Medium	Low
BuildPulse	Flaky detection only	Yes (stability)	Low	N/A
TestSplit (LPT)	Duration-based (LPT)	Yes	Low	Low

Table 2: Key Differences Between the Approaches

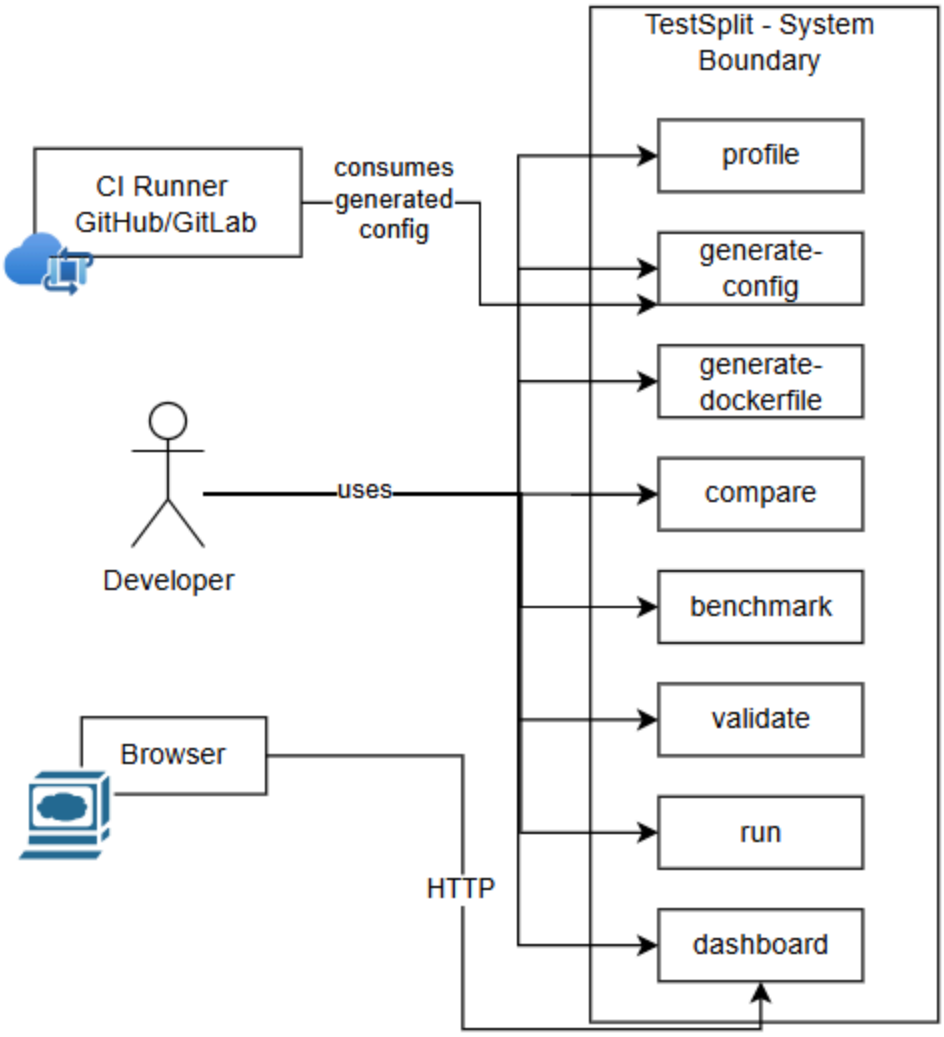

Figure 1: High-Level System Use Case Diagram

3. Design

TestSplit follows a layered architecture, separating concerns across parsing, profiling, scheduling, generation, and storage. Each layer has a single responsibility and communicates through well-defined interfaces, making the system testable and extensible [18], [19], [20]. The design prioritises auto-detection over explicit configuration. TestSplit infers container images, test dependencies, and lifecycle services from existing project files rather than requiring additional flags.

It is intentionally local-first, so all profiling data is stored as compressed JSON on disk, requiring no database or network connection and generated CI configurations are additive. TestSplit never modifies existing pipelines; it only produces new optimised job definitions alongside them.

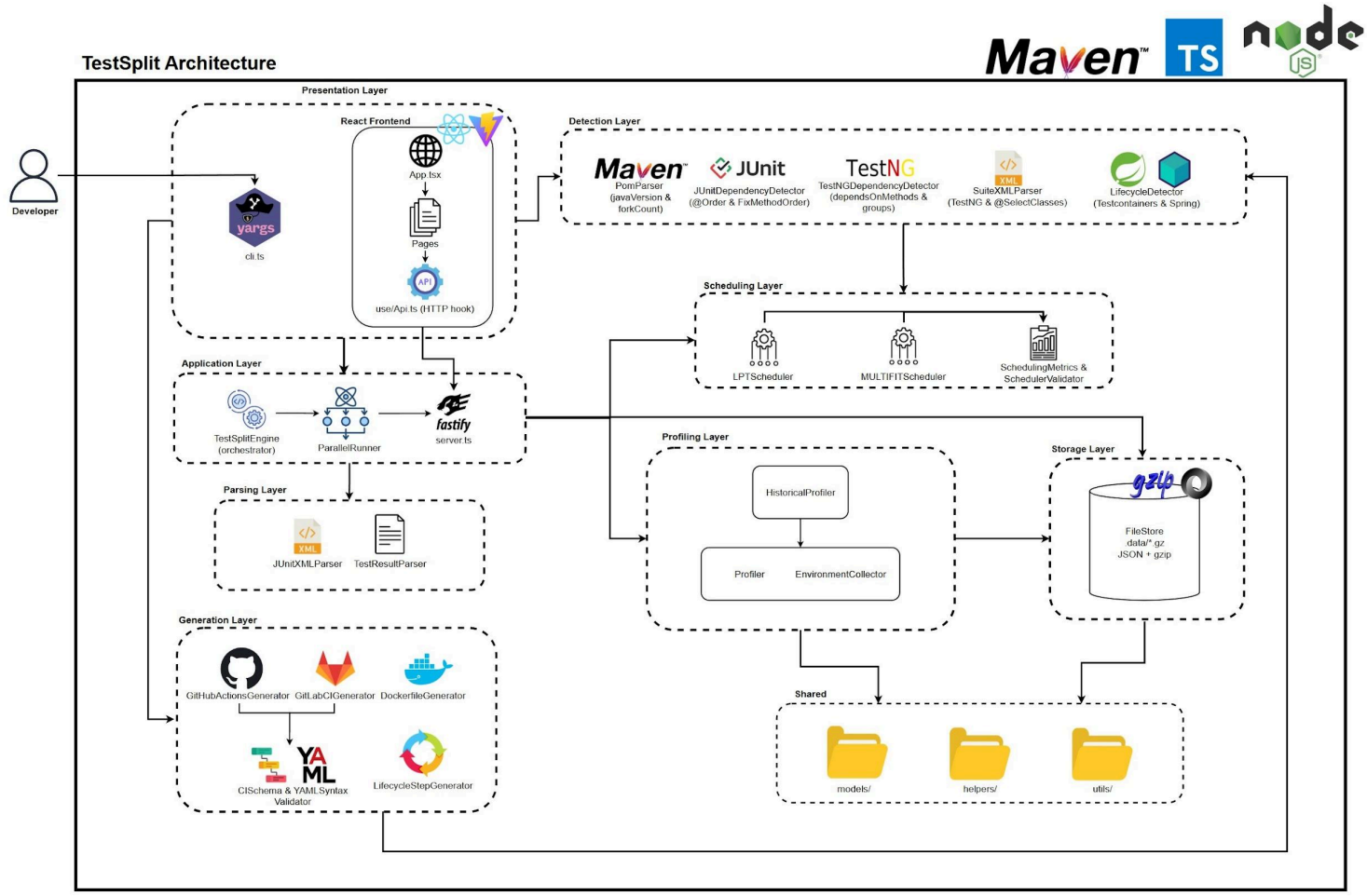

Figure 2: TestSplit System Architecture Diagram

3.1 System Architecture

TestSplit is organised into eight layers, as illustrated in Figure 2. The Presentation Layer exposes the system through two interfaces: A yargs-based CLI (cli.ts) and a React frontend served via a Fastify API. The Application Layer contains TestSplitEngine, which orchestrates the profile-and-schedule pipeline, and ParallelRunner, which handles parallel test execution [18], [19].

The Detection Layer analyses Java source files to extract Maven configuration, test ordering constraints, and lifecycle service requirements. The Scheduling Layer applies LPT or MULTIFIT to distribute tests across CI jobs. The Profiling Layer tracks historical test durations using exponential smoothing. The Generation Layer produces GitHub Actions and GitLab CI YAML, Dockerfiles, and lifecycle steps.

All persistent data is written through the Storage Layer via FileStore using gzip-compressed JSON. Shared models, helpers, and utilities underpin all layers.

3.2 Module Design

3.2.1 Test Result Parser

The Test Result Parser is the entry point for all test data in TestSplit. JUnitXMLParser reads Surefire XML reports produced by Maven [21] and, using fast-xml-parser [22], extracts individual test cases, their durations, class names, and pass/fail status.

TestResultParser normalises the raw parsed output into a consistent TestResult[] structure consumed by the Profiling Engine. The parser handles both JUnit 4 and JUnit 5 report formats [22], including nested <testsuite> elements and <testcase> entries with time attributes.

3.2.2 Profiling Engine

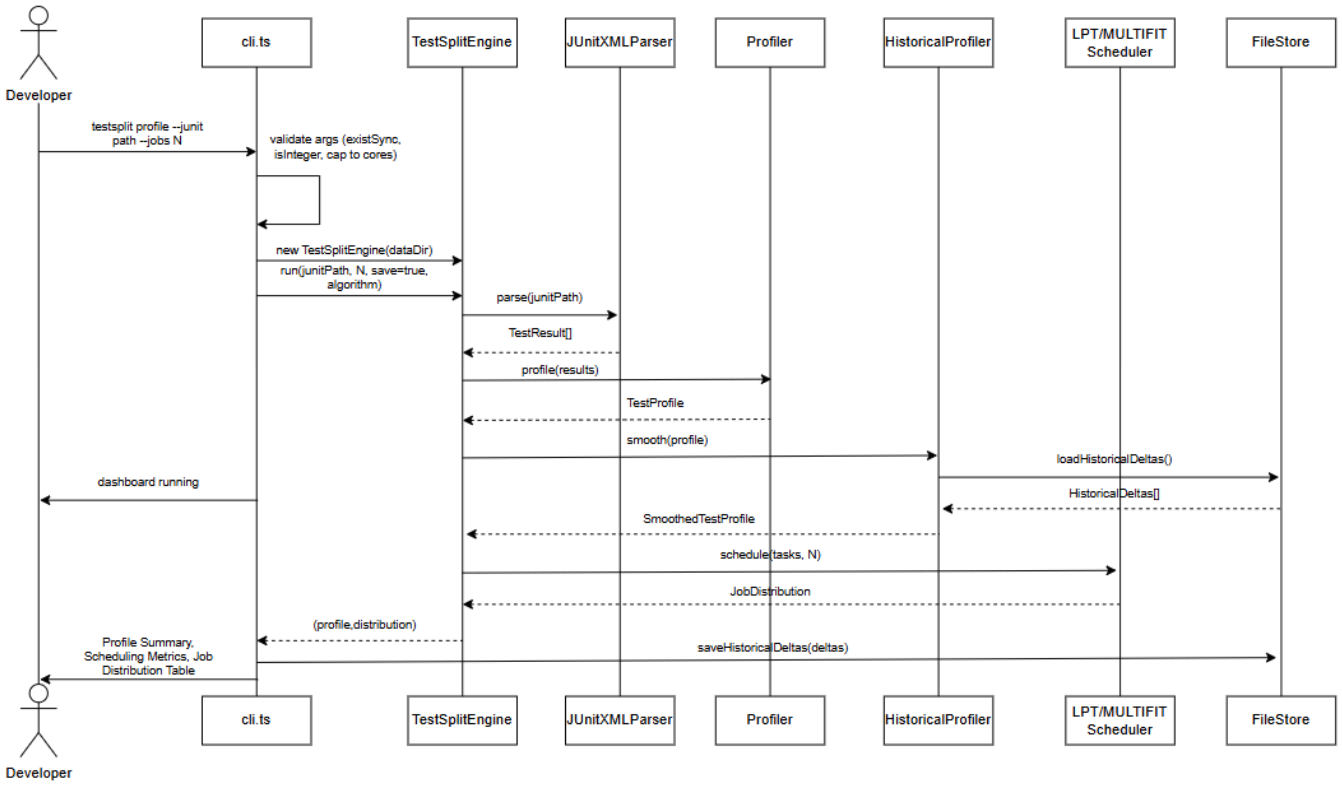

Figure 3: Profile Command - Sequence Diagram

The Profiling Engine is responsible for measuring and tracking test execution times across runs. Profiler receives the TestResult[] array from the parser and computes per-test statistics: mean duration, standard

deviation, and pass rate. EnvironmentCollector captures metadata at profiling time, including OS [23], CPU count, Node.js version, and git commit hash, ensuring profiles are environment-aware.

HistoricalProfiler extends this by applying exponential smoothing to each test's duration across runs [24], the design of this mechanism is described in detail in Section 3.4.

The output is a smoothed TestProfile containing per-test weighted durations, which are fed directly into the Scheduling Engine as Task[].

3.2.3 Scheduling Engine

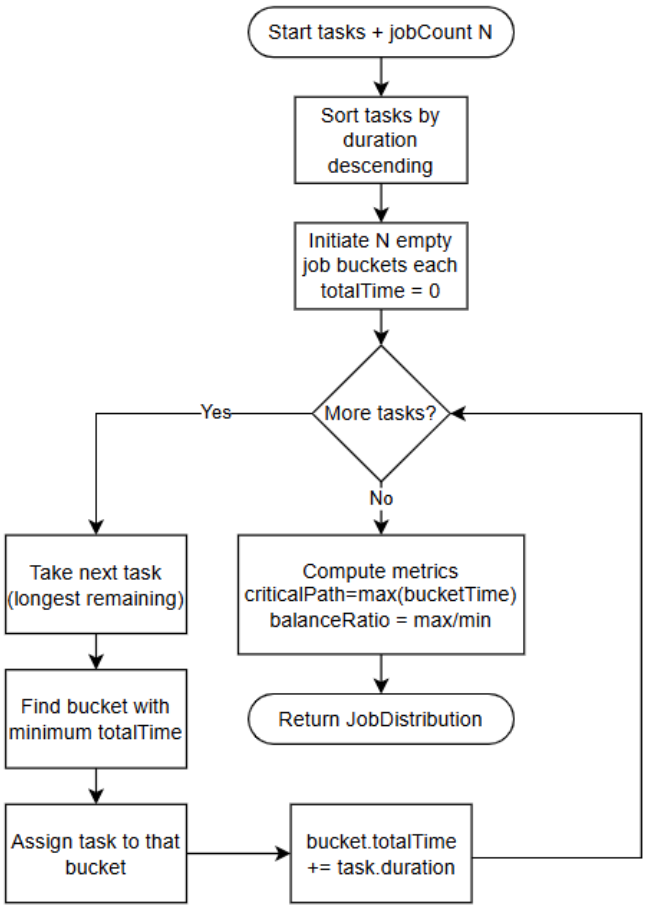

Figure 4: LPT Scheduling Data Flow Diagram L1 Diagram

The Scheduling Engine distributes tests across CI jobs to minimise total pipeline duration. It receives a Task[] array from the Profiling Engine, where each task carries an ID, smoothed duration, and an optional dependency list. Two scheduling algorithms are available, LPT and MULTIFIT [25], [26], both

implementing the same schedule(tasks, jobCount): JobDistribution interface, making them interchangeable.

When task dependencies are present, a topological sort is performed before scheduling to ensure that dependent tests are assigned in the correct order [27].

SchedulingMetrics computes post-scheduling statistics, makespan, balance ratio (min/max load) [5], critical path length, and estimated speedup over sequential execution. SchedulerValidator enforces preconditions before scheduling runs, rejecting invalid inputs early. The output is a JobDistribution passed to the Configuration Generator.

3.2.4 Configuration Generator

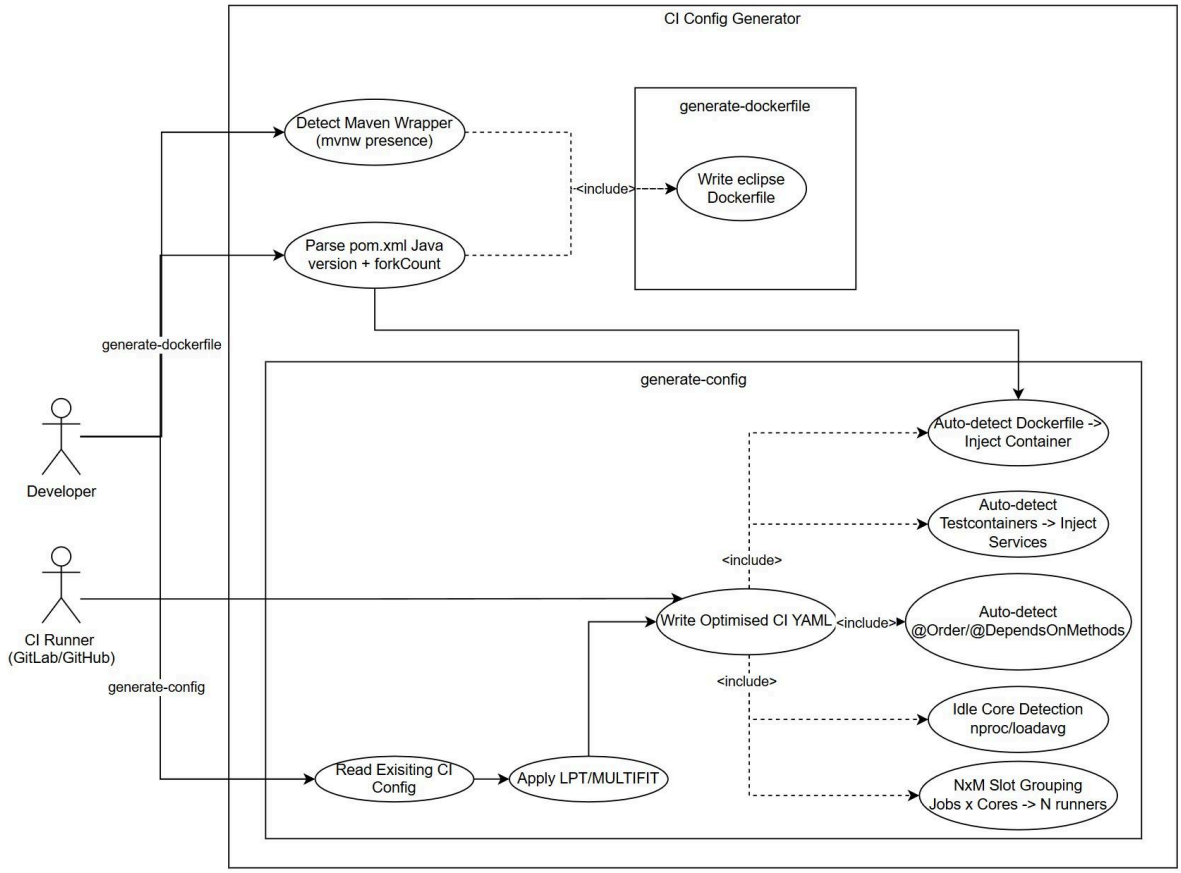

Figure 5: CI Configuration Generator - Use Case Diagram

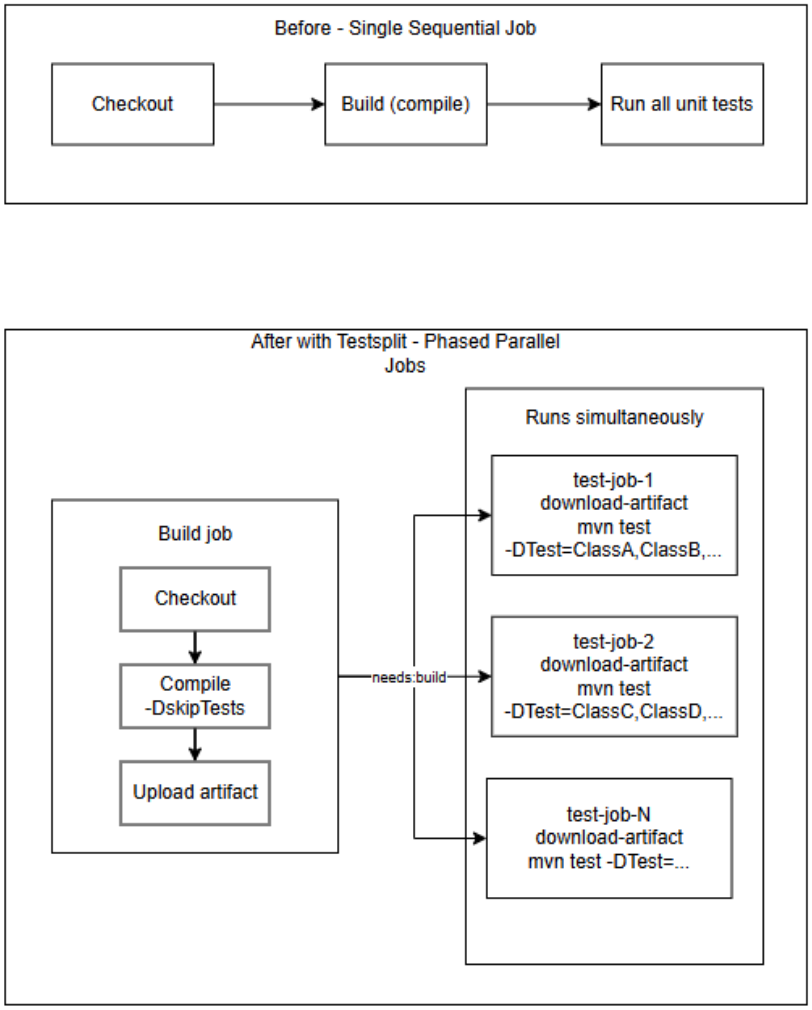

Figure 6: Before - After CI Pipeline Flow Diagram

The Configuration Generator translates a JobDistribution into platform-specific CI YAML. Two generators are provided, GitHubActionsGenerator and GitLabCIGenerator, both accepting the same inputs: job distribution, base job definition extracted from the existing CI config, container image, lifecycle services, and docker-compose flags [28], [29].

GitHubActionsGenerator produces a phased workflow with a build job followed by N parallel test jobs, each running a subset of tests. When a container image is detected, it is injected via the container: field. When Testcontainers or Spring services are detected, a services: block is added. GitLabCIGenerator follows the same pattern, using image: and services: fields per GitLab CI syntax [28], [30].

Both generators pass output through CISchemaValidator and YAMLSyntaxValidator before writing to disk, ensuring the generated configuration is syntactically and structurally valid. LifecycleStepGenerator

injects docker-compose startup and teardown steps when a docker-compose.yml is present in the project root [31].

Generated files are written alongside the existing CI configuration and never modify the original pipeline.

3.2.5 Data Storage

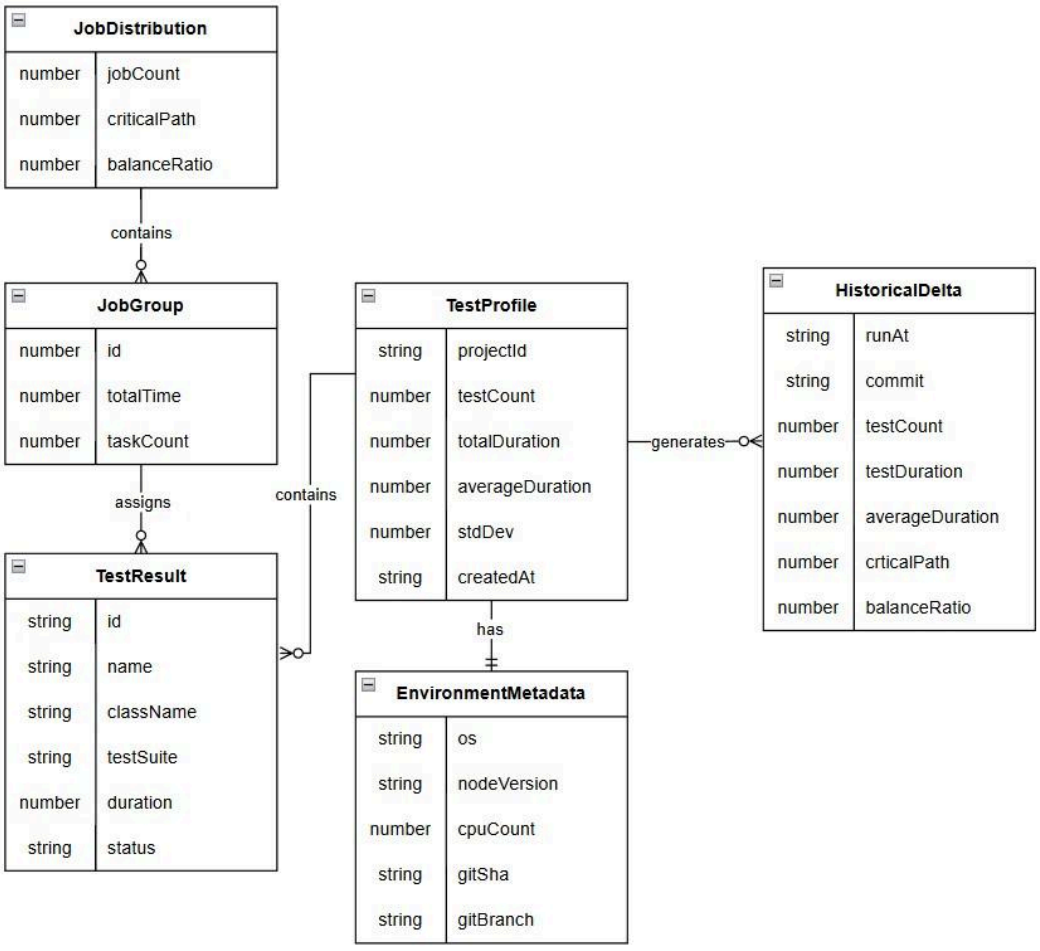

Figure 7: ER Diagram - Showing Attributes Stored

The Data Storage layer provides persistent storage for test profiles, historical deltas, and job distributions. All data is managed through FileStore, which reads and writes gzip-compressed JSON files under .data/ in the project directory. Compression is applied transparently, keeping storage overhead minimal for large historical datasets.

StoragePaths centralises all file path resolution, ensuring consistent naming across reads and writes. SchemaVersions tracks the version of each stored data structure, allowing forward-compatible migrations when the profile schema evolves between TestSplit versions [32]. Types and Interfaces define the storage contracts that the Profiling Engine and Scheduling Engine consume.

Historical data is stored across three categories: profiles (per-run test statistics), deltas (smoothed duration changes between runs), and distributions (job assignment history). This separation allows each to be queried independently by the Fastify API and dashboard [33].

No external database is required. The local-first design ensures TestSplit operates fully offline and without infrastructure dependencies.

3.2.6 User Interfaces

Figure 8: Wireframe of Overview Page (Phase 1)

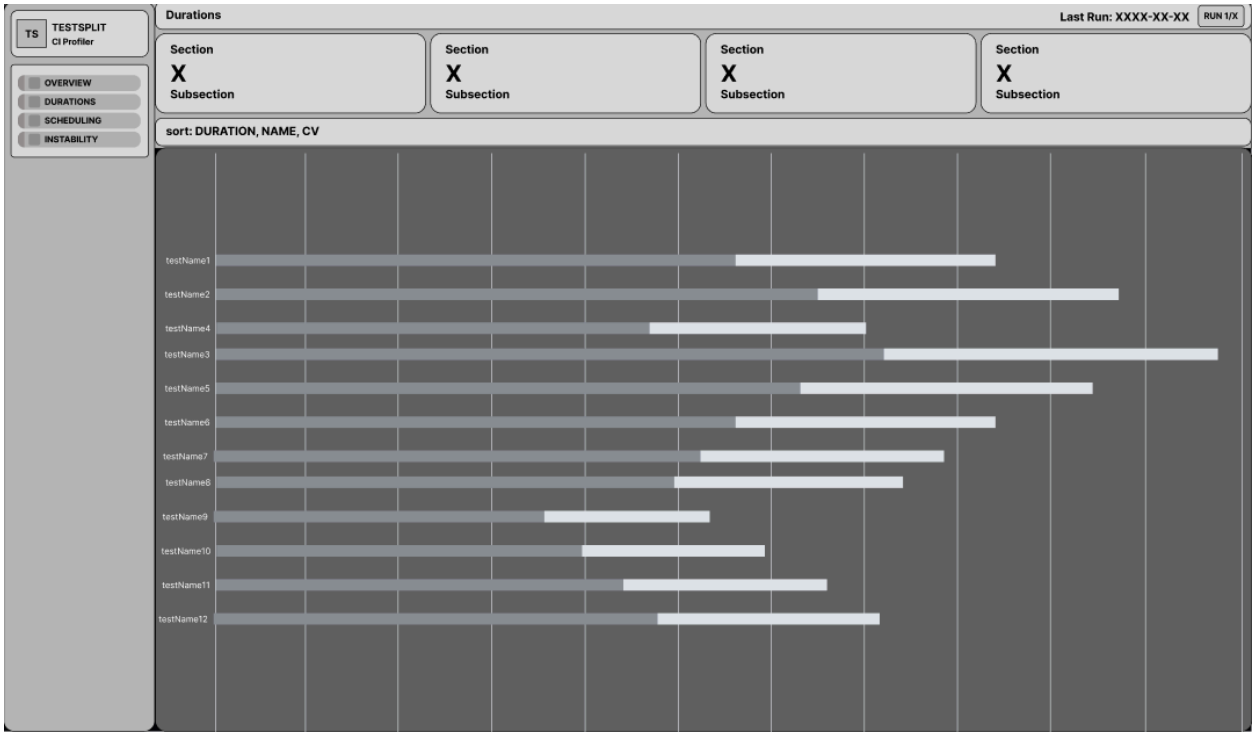

Figure 9: Wireframe of Durations Page (Phase 1)

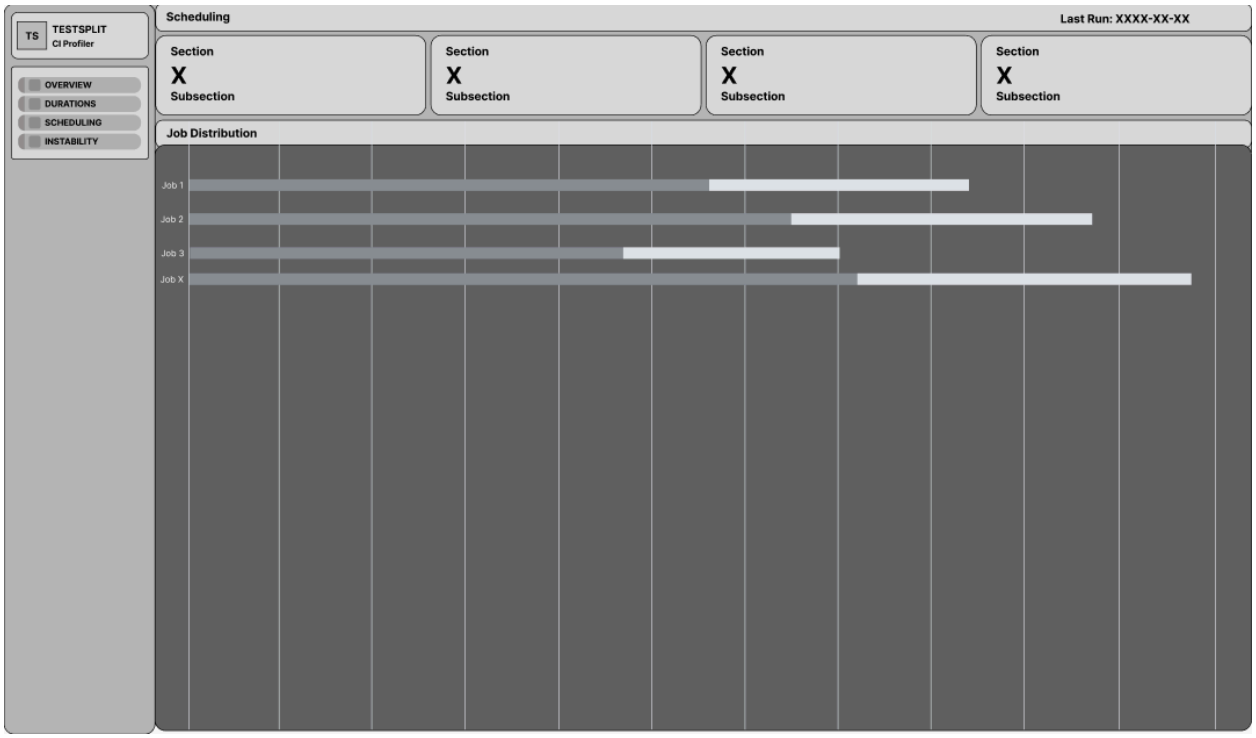

Figure 10: Wireframe of Scheduling Page (Phase 1)

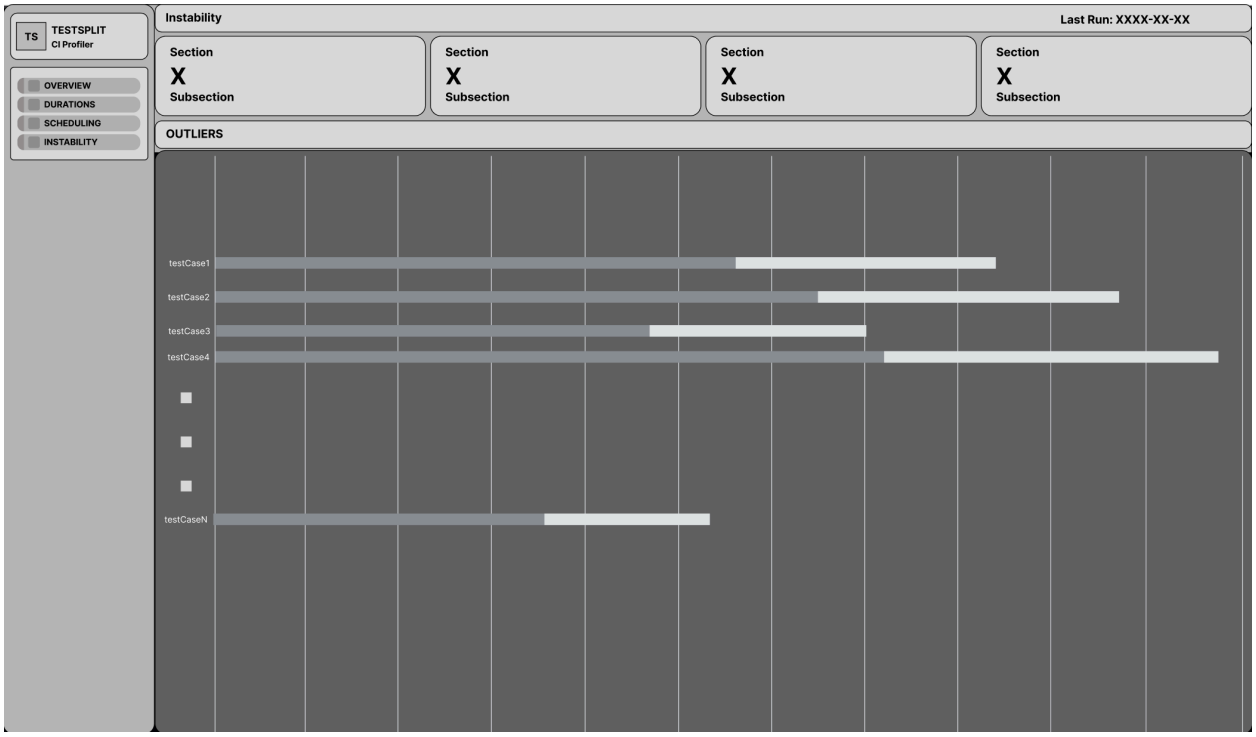

Figure 11: Wireframe of Instability Page (Phase 1)

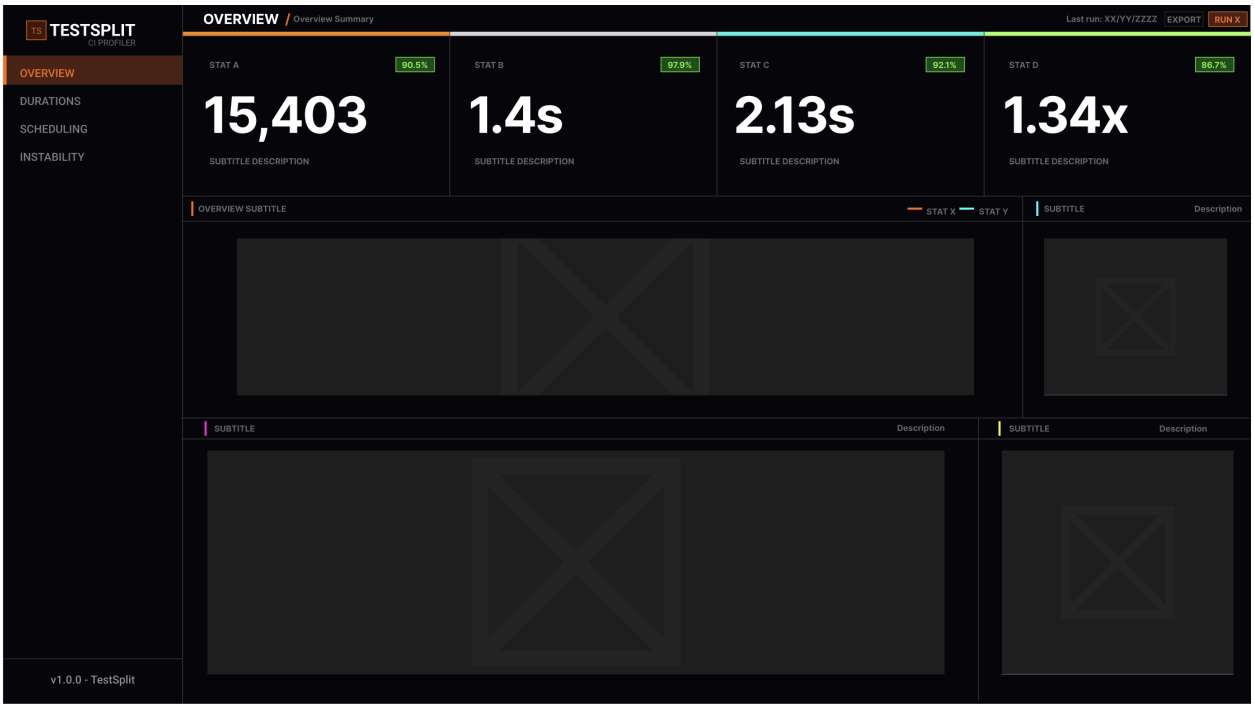

Figure 12: Updated Wireframe of Instability Page (Phase 2)

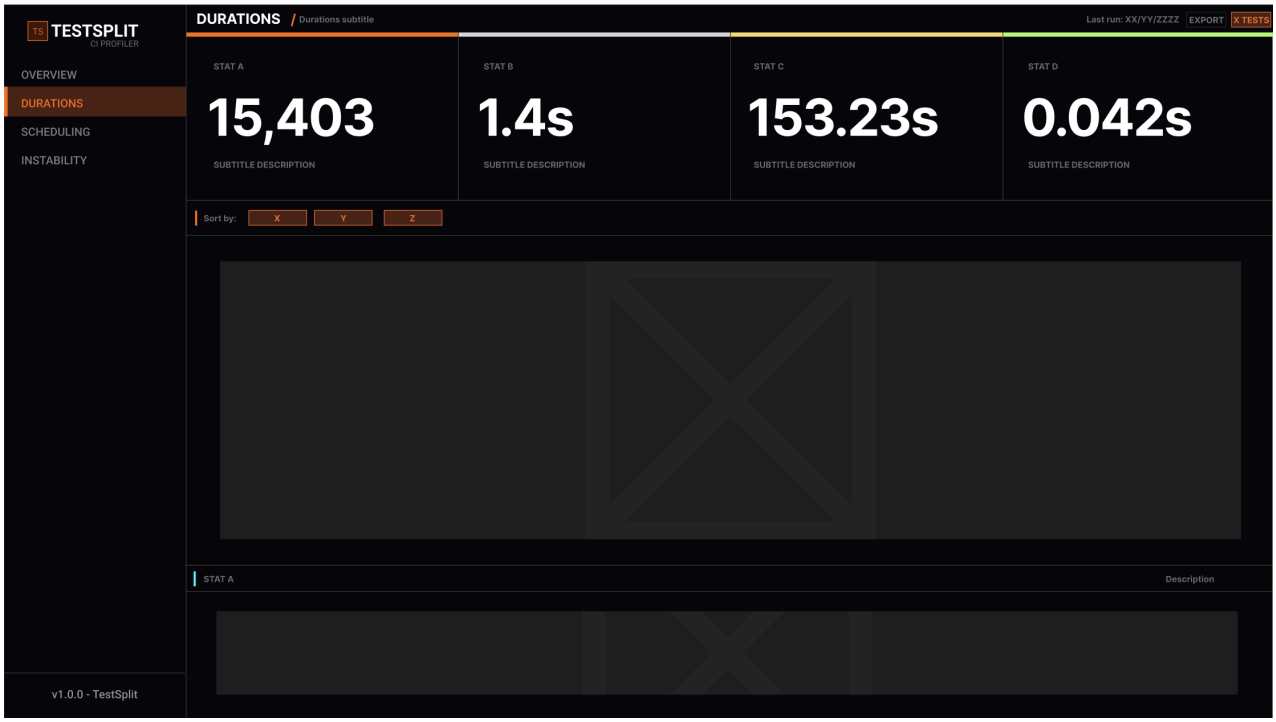

Figure 13: Updated Wireframe of Instability Page (Phase 2)

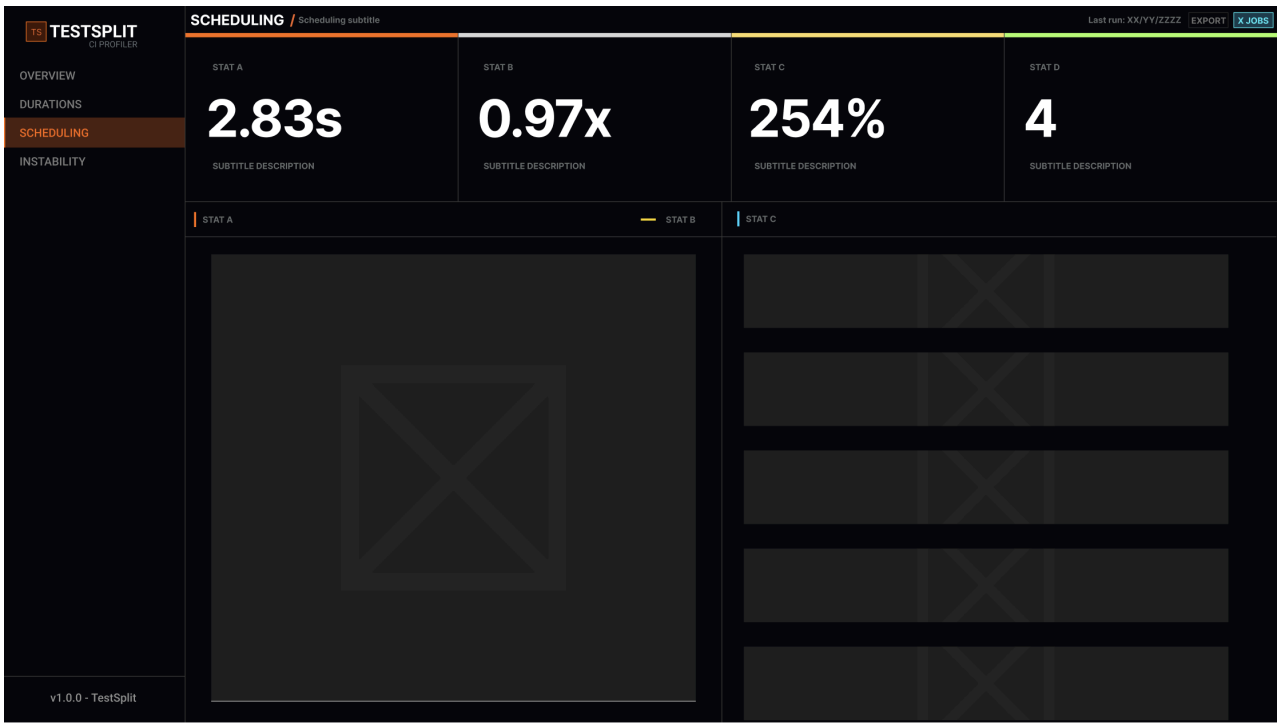

Figure 14: Updated Wireframe of Instability Page (Phase 2)

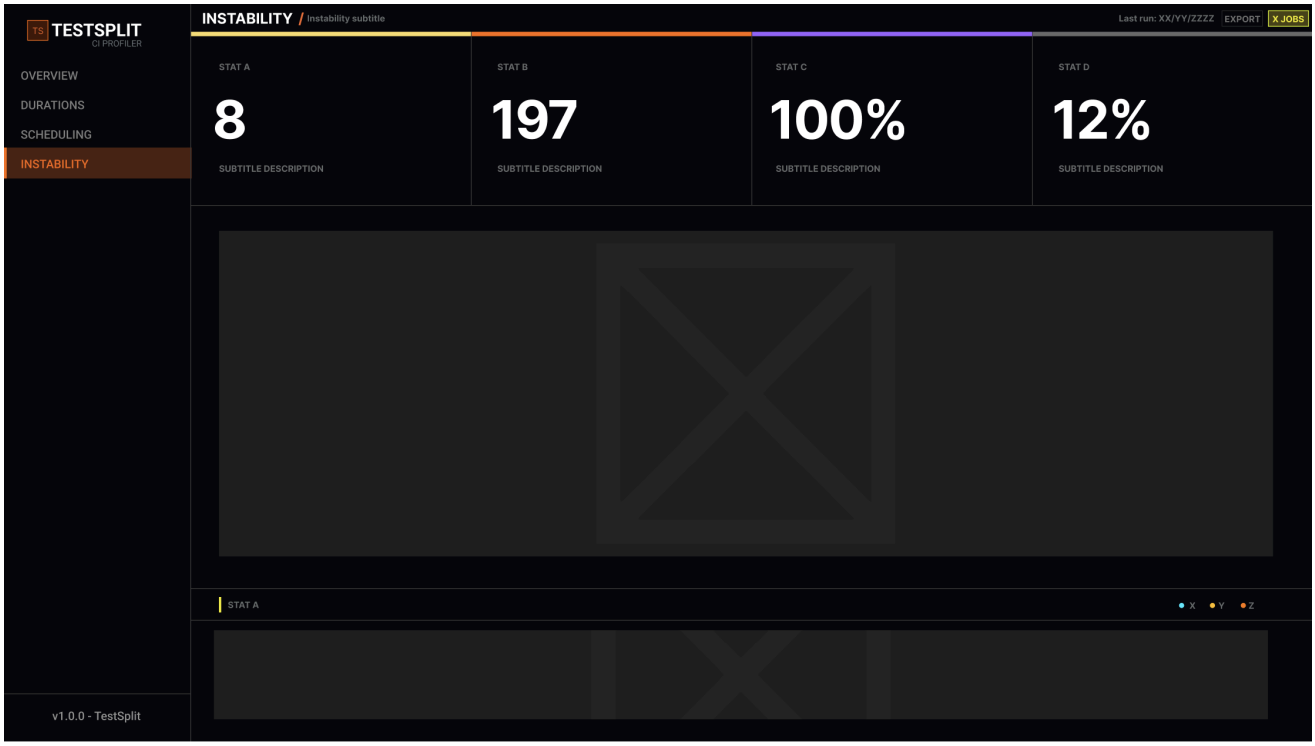

Figure 15: Updated Wireframe of Instability Page (Phase 2)

TestSplit exposes a command-line interface and a web dashboard.

The CLI is built with yargs and provides eight commands: profile, generate-config, generate-dockerfile, compare, benchmark, validate, run, and dashboard. Each command follows a consistent flag pattern with sensible defaults, minimising the developer's required input.

The web dashboard is served locally by a Fastify API on port 3001 and built with React, Vite, and Tailwind CSS. It provides four views: Overview, Durations, Scheduling, and Instability. The REST API and data fetching implementation are described in Section 4.2.8.

3.2.7 Dependency Detection

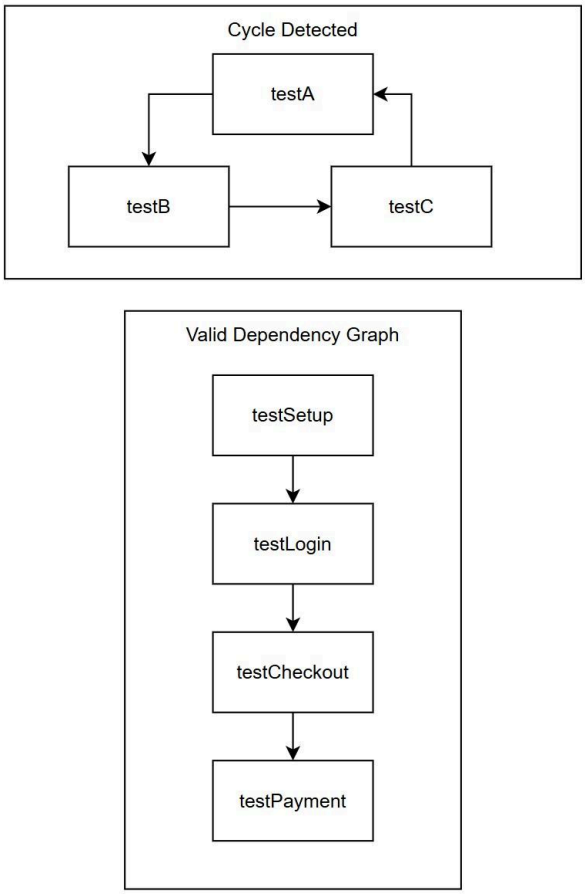

Figure 16: Valid Dependency Directed Graph

The Dependency Detection module analyses Java test source files to extract explicit test-ordering constraints and to build a dependency graph that is consumed by the Scheduling Engine.

The primary focus is JUnit. JUnitDependencyDetector supports both JUnit 5 @TestMethodOrder(OrderAnnotation.class) with @Order(n) annotations and JUnit 4 @FixMethodOrder(MethodSorters.NAME_ASCENDING). Only NAME_ASCENDING is supported for JUnit 4 [22], [35]. DEFAULT and JVM orderings are explicitly excluded as they are documented as non-deterministic.

The module was extended beyond the core JUnit scope to include TestNG, given its widespread use in Java enterprise projects and the prevalence of mixed or migrating codebases. TestNGDependencyDetector parses @Test(dependsOnMethods=...) and @Test(dependsOnGroups=...) annotations, resolving group membership to concrete task IDs [36]. SuiteXMLParser handles TestNG suite XML files and JUnit Platform @SelectClasses, extracting explicit class ordering from suite definitions.

Each detector outputs a Map<string, string[]> mapping each task ID to its dependencies. Results from all detectors are merged before being passed to the Scheduling Engine. If a cycle is detected in the dependency graph, scheduling is aborted, and a descriptive error is surfaced to the developer via the CLI. Detection runs automatically when src/test/java is present in the project root, requiring no additional flags.

3.2.8 Lifecycle Detection

The Lifecycle Detection module scans the project to identify external service dependencies required by the test suite and injects the appropriate startup configuration into the generated CI output. As illustrated in the system architecture diagram [Figure 2], LifecycleDetector resides in the Detection Layer while LifecycleStepGenerator resides in the Generation Layer, reflecting their distinct responsibilities: detection of service requirements and generation of CI output are deliberately separated.

LifecycleDetector performs two forms of detection. First, it scans Java source files under src/test/java for Testcontainers instantiation patterns, recognising:

1. PostgreSQLContainer
2. MySQLContainer
3. KafkaContainer
4. MongoDBContainer
5. RedisContainer

and maps each to its canonical Docker image and default port bindings [37], [38], [39], [40], [41]. Second, it detects Spring Boot test annotations, @EmbeddedKafka, @DataJpaTest, and @AutoConfigureTestDatabase, which imply external service requirements without explicit container declarations [42], [43], [44]. If a docker-compose.yml is present in the project root, compose-based lifecycle management takes precedence over individual service detection.

LifecycleStepGenerator translates the detected services into platform-specific CI configuration. For GitHub Actions, a services: block is injected into each test job. For GitLab CI, services are added as an array under the image: field. When docker-compose is detected, startup (docker compose up -d) [45] and teardown (docker-compose down) steps are injected before and after the test steps, respectively, with an if: always() condition in teardown to ensure cleanup regardless of the test outcome. Like dependency detection, lifecycle detection runs automatically with no additional flags required.

3.2.9 Parallel Runner

The Parallel Runner handles the actual execution of test subsets across multiple processes when the run command is invoked. It operates on a JobDistribution produced by the Scheduling Engine, spawning a child process for each job and managing their concurrent execution [46]. As shown in the system architecture diagram [Figure 2], the Parallel Runner sits in the Application Layer alongside TestSplitEngine, reflecting its role as a runtime execution component rather than a configuration or analysis concern.

WorkQueue maintains a pool of pending tasks and feeds them to available workers, implementing a work-stealing strategy in which idle workers pull tasks from the queues of busier workers to prevent underutilisation [47]. CoreAffinity pins each worker process to a specific CPU core on Linux using taskset, reducing context-switching overhead and improving timing consistency across runs [48]. TimingFeedback captures real execution times from each worker and feeds them back to HistoricalProfiler on completion, allowing future scheduling decisions to be informed by actual observed durations rather than estimates alone.

Together, these components form a closed feedback loop: the Scheduling Engine distributes work based on historical profiles, the Parallel Runner executes and measures it, and the Profiling Engine updates the historical record for the next run.

3.3 Interaction Flow

This section describes the step-by-step operation of TestSplit's primary profiling workflow. The generate-config workflow is covered in Section 3.2.4

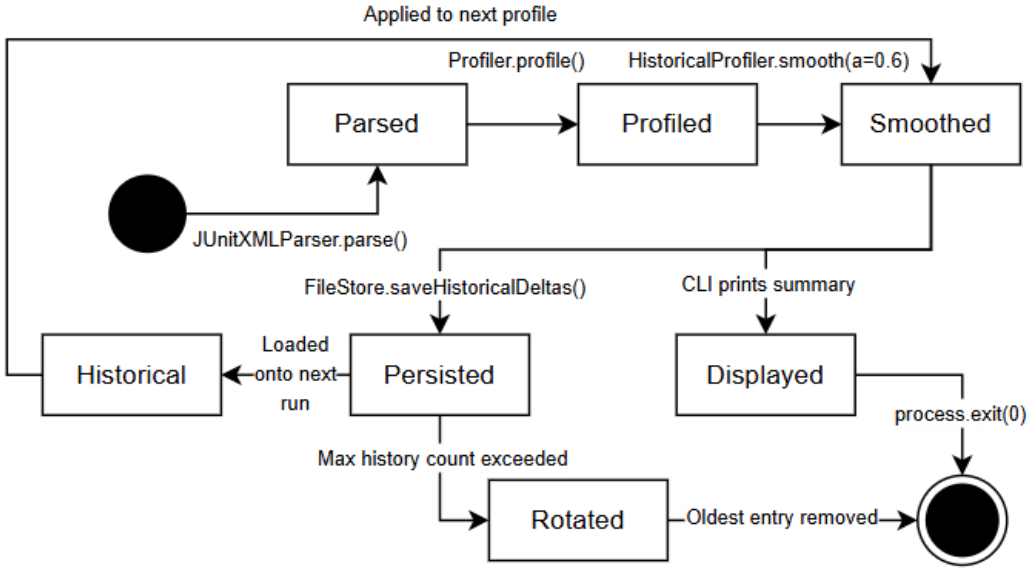

Figure 17: Profile Data Flow Diagram

Profile Workflow

1. The developer invokes testsplit profile --junit <path> --jobs N
2. The CLI validates arguments and instantiates TestSplitEngine
3. JUnitXMLParser parses the Surefire XML report, producing TestResult[]
4. Profiler computes per-test statistics, including mean duration and standard deviation
5. EnvironmentCollector captures system metadata at profiling time

6. HistoricalProfiler applies exponential smoothing against previous runs loaded from FileStore, flags unstable tests, and persists the updated profile.
7. The LPT or MULTIFIT scheduler distributes tasks into N jobs, producing a JobDistribution.
8. SchedulingMetrics computes the makespan, balance ratio, and speedup estimate [5].
9. The CLI prints a profile summary and scheduling metrics table to the terminal.

The profile data lifecycle state diagram [Figure 17] illustrates the state transitions of a test profile from initial creation through smoothing, instability detection, and persistence.

3.4 Historical Profiling Design

The historical profiling system is designed to improve scheduling accuracy over time by learning from past test runs rather than relying on the timings of a single execution.

3.4.1 Exponential Smoothing

Rather than averaging all historical durations equally, TestSplit applies exponential smoothing with a smoothing factor of $\alpha=0.6$ to each test's duration across runs. Each new observation is weighted more heavily than older ones, allowing the profiler to remain responsive to genuine performance changes while dampening noise from one-off slow tests. The smoothed duration for a test at run t is computed as:

$$S(t) = \alpha * x(t) + (1-\alpha) * S(t-1)$$

where $x(t)$ is the observed duration and $S(t-1)$ is the previous smoothed value. A value of $\alpha=0.6$ was selected to balance recency bias against stability [24].

3.4.2 Instability Detection

A test is flagged as unstable when its standard deviation across recent runs exceeds a defined threshold relative to its mean duration. Unstable tests are highlighted in the CLI output to alert the developer that their timing contribution to scheduling may be unreliable.

3.4.3 Regression Detection

HistoricalProfiler computes deltas between successive smoothed profiles and flags tests whose durations exceed a configurable threshold. These regression flags are stored alongside the profile and surfaced in the dashboard's Instability view.

3.4.4 Storage Detection

Each profile run is persisted as a gzip-compressed JSON file under .data/profiles/. Historical deltas are stored separately under .data/historical/, allowing them to be queried independently. SchemaVersions ensures forward compatibility as the profile structure evolves between TestSplit versions.

3.5 Validation Design

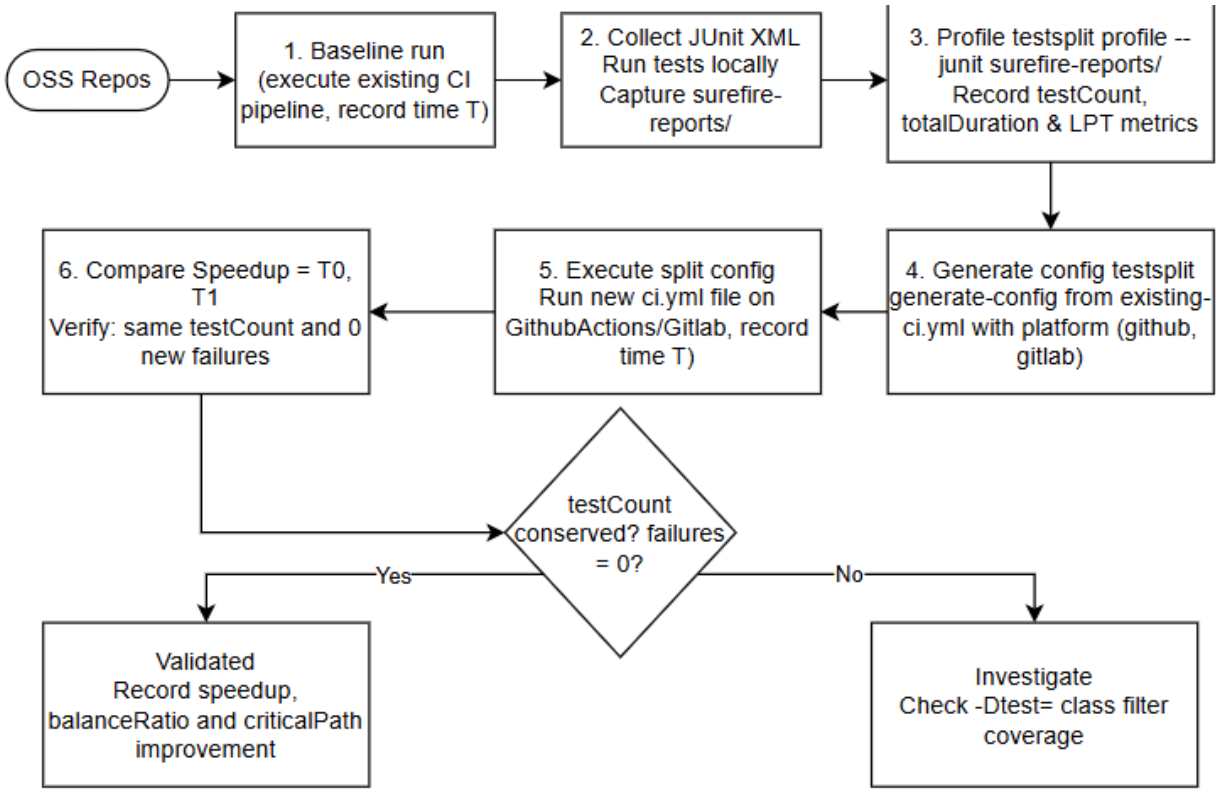

Figure 18: Validation Methodology Flow Diagram

TestSplit validates generated CI configurations before writing them to disk, ensuring that output is both syntactically correct and structurally compliant with the target platform's schema.

3.5.1 YAML Syntax Validation

YAMLSyntaxValidator parses the generated YAML string using a standards-compliant parser before output. Any syntax error, mismatched indentation, invalid characters, or malformed structure is caught at this stage and reported to the developer with a descriptive message. This prevents invalid files from being written to the project.

3.5.2 Schema Validation

CISchemaValidator defines a common interface that platform-specific validators implement. Each validator checks the generated configuration against the structural requirements of its target platform, verifying that required fields are present, job names are valid, and step definitions conform to expected formats for GitHub Actions and GitLab CI, respectively. getSchemaValidator selects the appropriate validator based on the target platform.

3.5.3 Input Validation

Beyond generated output, TestSplit validates CLI inputs at the point of entry. File paths are checked for existence, job counts are verified as positive integers and capped at the number of available CPU cores, and algorithm flags are validated against the set of supported values. Validation errors are surfaced immediately with actionable messages, rather than silently failing mid-execution.

3.5.4 Dependency Cycle Validation

As part of dependency-aware scheduling, the dependency graph is validated for cycles before scheduling proceeds. As described in Section 3.2.7, the dependency graph is validated for cycles before scheduling proceeds, aborting the scheduling and surfacing an error to the CLI if any are found.

3.6 Docker & CI Integration Design

TestSplit integrates with Docker and CI platforms at two levels: generating containerised job definitions for consistent test execution environments, and producing Dockerfile configurations for Java projects.

3.6.1 Container Image Injection

When a Dockerfile is present in the project root, TestSplit extracts the base image and injects it into the generated CI job definitions. For GitHub Actions, this is applied via the container: field, ensuring all test jobs run inside the same Docker image used locally. For GitLab CI, the image is applied via the image field at the job level. This guarantees environment consistency between local development and CI execution, eliminating the "works on my machine" class of test failures.

3.6.2 Dockerfile Generation

The generate-dockerfile command produces a Dockerfile tailored to the project's Java version, extracted from pom.xml via PomParser. The base image uses eclipse-temurin:<version>-jdk, the canonical OpenJDK Docker image maintained by the Adoptium project, chosen over the deprecated openjdk image for long-term support and security maintenance [49]. When a Maven wrapper (mvnw) is detected, it is used in preference to a system Maven installation. Docker layer caching is enabled via the dependency:go-offline flag to avoid re-downloading dependencies on every build [50].

3.6.3 Lifecycle Service Injection

Lifecycle service injection is handled by LifecycleStepGenerator as described in Section 3.2.8.

3.6.4 CI Platform Targeting

Generated configurations are written as separate files and never modify existing pipelines. GitHub Actions output follows the workflow syntax specification, GitLab CI output follows the YAML syntax reference. Both are validated before being written to disk as described in Section 3.5.

4. Implementation

4.1 Development Environment

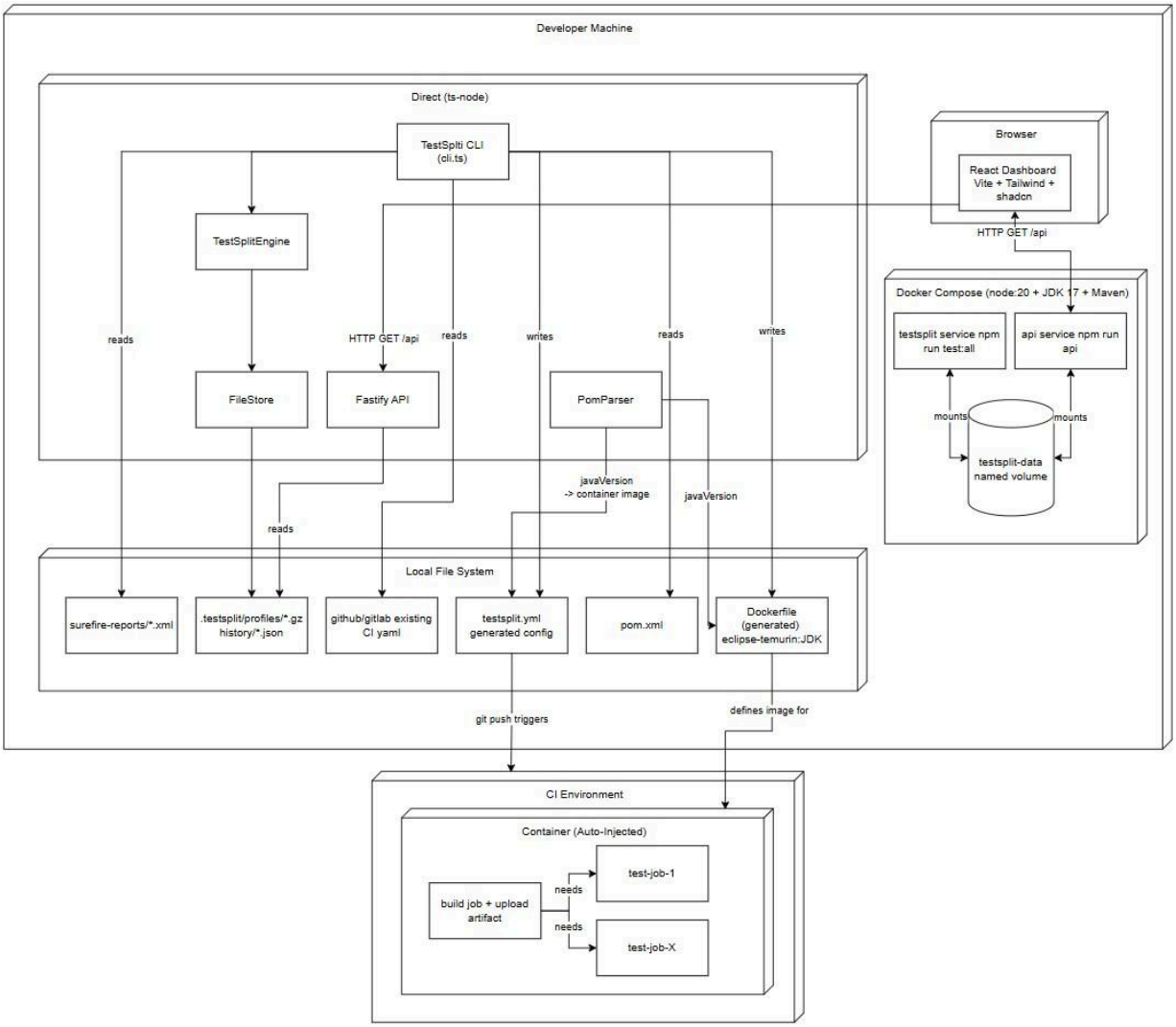

Figure 19: Local Deployment Diagram

TestSplit is developed as a single-package TypeScript project targeting Node.js 18+. The backend is written entirely in TypeScript and compiled with tsc; Jest and ts-jest are used for unit and integration testing. The frontend is a separate Vite project under src/frontend/, built with React, Tailwind CSS, and shadcn/ui; Vitest and React Testing Library are used for frontend unit and component testing.

Languages and Runtimes

- yargs: CLI argument parsing
- Fastify: REST API server
- React 18 + Vite + Tailwind CSS + shadcn/ui + Recharts: web dashboard
- SWR: data fetching and cache management in the React dashboard
- fast-xml-parser: JUnit XML and TestNG suite XML parsing
- Jest + ts-jest: backend unit and integration testing
- Vitest + @testing-library/react: frontend unit and component testing
- @vitest/coverage-v8: frontend code coverage reporting
- Husky: git hook management (pre-commit, pre-push)
- js-yaml: YAML parsing and generation

CI Platforms Targeted

- GitHub Actions
- GitLab CI

Operating System Requirements

TestSplit runs on macOS, Linux, and Windows. Core affinity pinning via taskset is Linux-only, on other platforms, CoreAffinity degrades gracefully without thread pinning.

4.2 Key Features Implemented

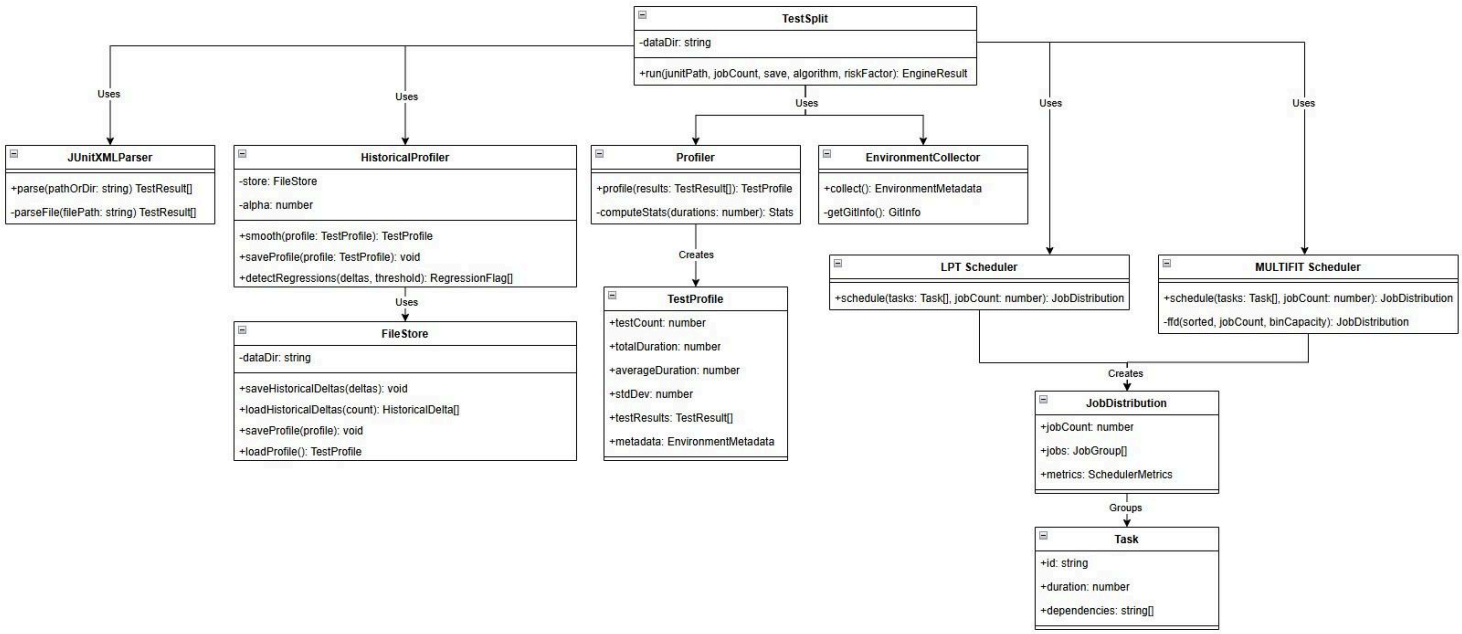

Figure 20: High-Level Class Diagram

4.2.1 JUnit XML Parsing

JUnit XML parsing is implemented using fast-xml-parser with strict attribute handling enabled, ensuring time attributes on <testcase> elements are correctly extracted as numeric values rather than strings. The parser traverses nested <testsuite> elements recursively, handling both flat and hierarchical Surefire report structures produced by JUnit 4 and JUnit 5, respectively.

TestResultParser normalises the raw parsed output by coercing duration values to milliseconds, stripping package prefixes from class names where needed, and mapping Surefire failure and error elements to a unified status field. Handled edge cases include missing time attributes (defaulting to 0), empty test suites, and malformed XML that would otherwise cause the parser to throw an exception.

The parser accepts either a single XML file path or a directory, scanning recursively for all *.xml files that match the Surefire report naming convention when a directory is provided.

Surefire XML reports expose test name, class name, duration, and pass/fail status. Metrics such as CPU usage, memory consumption, test categories, thread assignment, and retry counts are not included in the format. TestSplit scheduling decisions are therefore based on historical duration, which is the most reliable and universally available signal across Java test suites. Extending profiling granularity beyond duration would require instrumentation outside the Surefire report format, such as JVM agents or custom test listeners.

4.2.2 Test Profiling & Historical Tracking

Test profiling is implemented across three components: Profiler, EnvironmentCollector, and HistoricalProfiler. On each invocation, Profiler iterates over the TestResult[] array and computes per-test statistics, including mean duration, standard deviation, and pass rate. EnvironmentCollector captures a snapshot of the runtime environment: OS platform, CPU count, Node.js version, and current git commit hash, attached to the profile for environment-aware comparison across runs.

HistoricalProfiler implements the exponential smoothing update loop, loading the previous smoothed values from the FileStore and applying the formula $S(t) = \alpha * x(t) + (1-\alpha) * S(t-1)$ for each test. If no prior value exists for a test (first run), the observed duration is used as the initial smoothed value. Instability is flagged by computing the coefficient of variation across the last N stored deltas for each test and comparing against a fixed threshold. Regression is detected by comparing the current smoothed value with the previous one and flagging tests whose delta exceeds a configurable percentage threshold.

The full interaction between these components during a profile command invocation is illustrated in the profile command sequence diagram (Figure 2).

4.2.3 CI Configuration Generation

CI configuration generation is implemented across two platform-specific generators, GitHubActionsGenerator and GitLabCIGenerator, both accepting a shared input interface containing the job distribution, base job definition, container image, detected services, and docker-compose flags.

The existing CI configuration is parsed on invocation to extract the base job definition and test command. In GitHub Actions, the .github/workflows directory is scanned for workflow files that contain test jobs. For GitLab CI, .gitlab-ci.yml is read directly. The extracted base job serves as the template for each generated test job, preserving environment variables, caches, and runner tags defined in the original pipeline.

Each generator builds N test jobs from the JobDistribution, injecting the appropriate test subset command into each job's run step. Job naming follows a consistent test-split-N convention. When a container image is present, it is injected at the job level. When services are detected, they are added as a services: block. When docker-compose is detected, startup and teardown steps are prepended and appended, respectively.

Output is passed through YAMLSyntaxValidator and CISchemaValidator before being written to disk as testsplit.yml. The before-and-after CI pipeline flow diagram (Figure 5) illustrates the structural difference between the original unoptimised pipeline and the TestSplit-generated output.

4.2.4 Dockerfile Generation

Dockerfile generation is implemented in DockerfileGenerator, which accepts a Java version string and a boolean indicating whether a Maven wrapper is present. Both inputs are supplied by PomParser, which extracts the Java version from the <java.version> or <maven.compiler.source> properties in pom.xml and checks whether mvnw exists in the project root.

The generated Dockerfile uses eclipse-temurin:<version>-jdk as the base image, sourced from the official Adoptium Docker Hub image. Dependencies are pre-fetched using mvn dependency:go-offline in a dedicated layer before copying the full source, ensuring Docker layer caching is preserved across builds where only source files change [51]. When a Maven wrapper is detected, ./mvnw is used in place of mvn throughout the Dockerfile.

The output is written to the path specified by the --out flag, defaulting to Dockerfile in the project root if not provided.

4.2.5 Dependency Detection

Detection is implemented using three detectors, each scanning Java source files and outputting a Map<string, string[]> mapping task IDs to their dependencies. Results are merged into a single dependency map before being passed to the Scheduling Engine.

JUnitDependencyDetector scans for @TestMethodOrder(OrderAnnotation.class) at the class level and collects all @Order(n) annotations on test methods within that class. Methods are sorted by their order value and chained as sequential dependencies, each method depending on the previous. For JUnit 4, classes annotated with @FixMethodOrder(MethodSorters.NAME_ASCENDING) are detected and their test methods extracted and sorted alphabetically before chaining.

TestNGDependencyDetector parses @Test annotations for dependsOnMethods and dependsOnGroups attributes, handling both single string and array syntax. A group index is built from all group declarations, allowing dependsOnGroups references to be resolved to concrete task IDs before the dependency map is populated.

SuiteXMLParser parses TestNG suite XML files using fast-xml-parser and extracts class ordering from <test>/<classes> blocks. For the JUnit Platform Suite, @SelectClasses annotations are parsed from source files, with import alias resolution, to handle fully qualified and aliased class references.

The merged dependency map is validated for cycles using a depth-first topological sort before being passed to the scheduler. The dependency-directed graph (Figure 11) illustrates a valid dependency graph produced by the detector.

4.2.6 Lifecycle Detection & Service Injection

Lifecycle detection is implemented in LifecycleDetector using regular expression pattern matching against Java source file content. Each source file under src/test/java is read and matched against two sets of patterns: Testcontainers container class instantiations and Spring Boot test annotations.

Testcontainers detection matches new <ContainerClass>(...) expressions using a map of known container class names to their canonical service definitions. Each match produces a ServiceRequirement containing the service type, Docker image, default port bindings, and required environment variables. Spring Boot detection matches annotation names directly @EmbeddedKafka, @DataJpaTest, and @AutoConfigureTestDatabase and maps each to its implied service requirement.

Results are deduplicated by service type, ensuring that a postgres service detected from multiple test files appears only once in the output. If a docker-compose.yml is present in the project root, individual service detection is bypassed entirely and compose-based lifecycle management is used instead.

LifecycleStepGenerator translates the deduplicated ServiceRequirement[] into platform-specific CI output. For GitHub Actions, buildGitHubServices produces a services block keyed by service type. For GitLab CI, buildGitLabServices returns an array of image strings. When docker-compose is active, buildDockerComposeStartStep and buildDockerComposeStopStep produce the corresponding step definitions, with if: always() applied to the teardown step to guarantee cleanup regardless of test outcome.

4.2.7 Parallel Test Execution

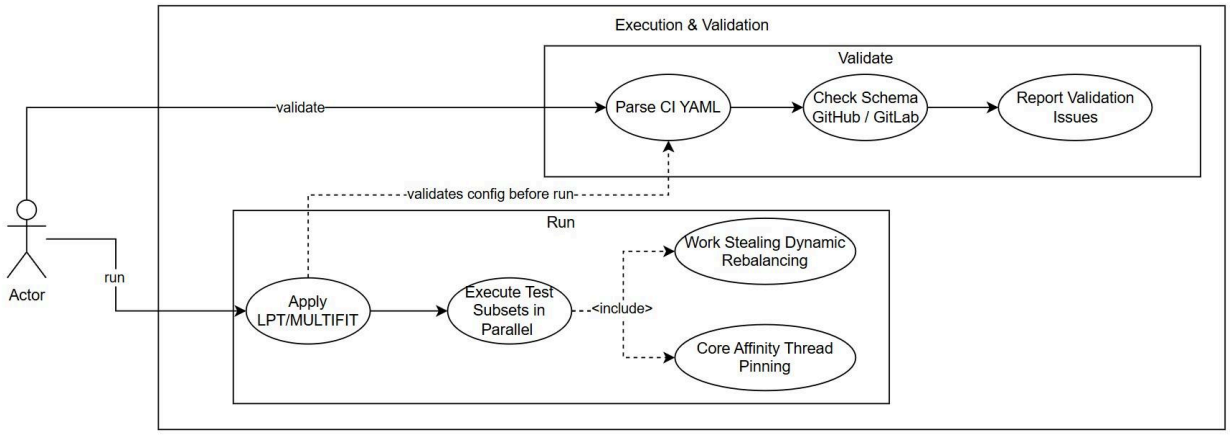

Figure 21: Execution & Validation Use Case Diagram

Parallel test execution is implemented across four components in src/backend/runner/. On invocation of the run command, ParallelRunner receives the JobDistribution from the Scheduling Engine and spawns a child process for each job using Node.js child_process.spawn, passing the test subset command and environment variables for each job.

WorkQueue maintains a shared queue of pending tasks across all workers. When a worker completes its assigned tasks ahead of others, it queries WorkQueue for remaining tasks from busier workers, implementing work stealing to prevent idle cores from sitting unused, while the slowest job determines the overall makespan.

CoreAffinity uses taskset on Linux to pin each spawned process to a dedicated CPU core, reducing operating system context-switching overhead and improving timing consistency between runs. On macOS and Windows, CoreAffinity detects platform incompatibility and skips pinning gracefully without affecting execution.

TimingFeedback intercepts the completion signal from each worker, records the actual wall-clock duration of each test subset, and passes the observed timings back to HistoricalProfiler. This closes the feedback loop between execution and scheduling; real durations observed during run are incorporated into the smoothed profile used for future profile and generate-config invocations.

Node.js worker_threads was considered for parallelism but rejected in favour of child_process.spawn. Worker threads share the same process memory and are subject to the Node.js event loop, making them unsuitable for running isolated test processes with independent JVM instances. CI-level parallelism via spawned child processes provides true process isolation, matching the environment each test job would run in on an actual CI runner.

The execution and validation use case diagram [Figure 21] illustrates the actor interactions with the run and validate commands.

4.2.8 Web Dashboard & Rest API

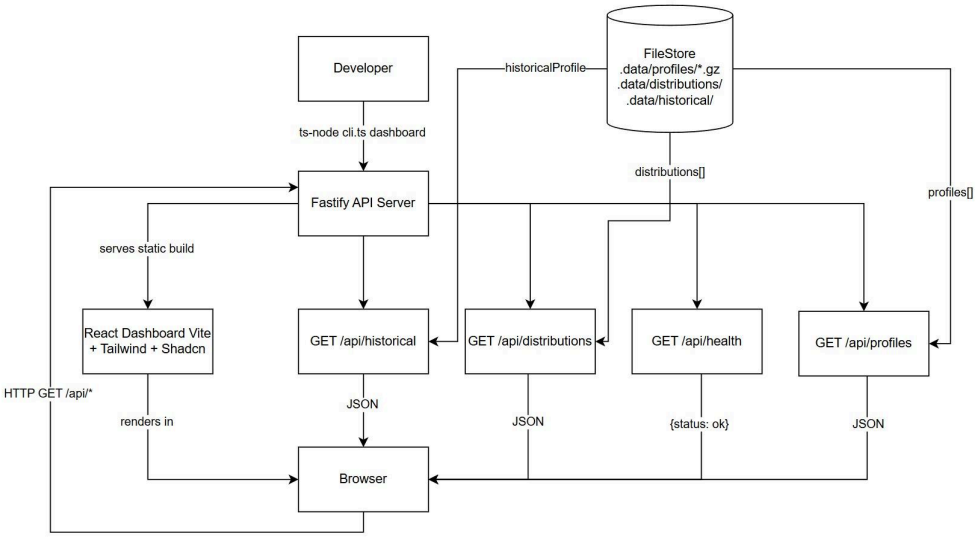

Figure 22: Frontend Data Flow Diagram Level 1

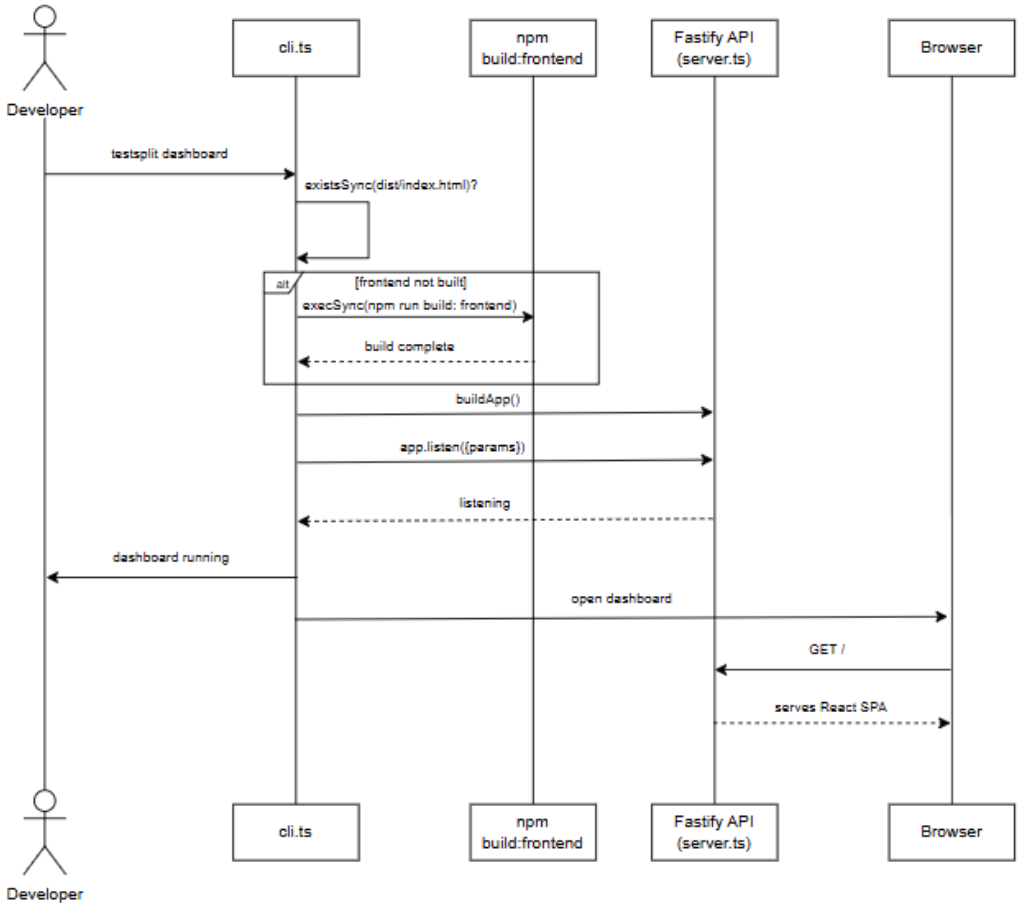

Figure 23: Dashboard Command Sequeunce Diagram

The web dashboard is implemented as a Fastify REST API serving a React single-page application. The API server is started by the dashboard CLI command, which builds the frontend if no production build exists in src/frontend/dist/, then starts the Fastify server on port 3001 and automatically opens the browser unless --no-open is passed.

The REST API exposes five endpoints:

1. GET /api/health: returns server status
2. GET /api/summary: returns aggregate test statistics across all stored profiles
3. GET /api/tests: returns per-test statistics with sorting and pagination support
4. GET /api/jobs: returns job distribution history
5. GET /api/trends: returns historical smoothed duration trends per test

All endpoints read directly from FileStore, requiring no database. Responses are JSON and consumed by the frontend via the useApi hook, which wraps SWR for request deduplication, caching, and loading and error state management per page.

The React frontend is built with Vite and provides four pages: Overview, Durations, Scheduling, and Instability, each fetching data from the relevant API endpoint and rendering it using Recharts. PageLoadingSkeleton and PageErrorState components handle loading and error states consistently across all pages.

The frontend DFD [Figure 23] and dashboard command sequence diagram [Figure 24] illustrate the data flow and command lifecycle for the dashboard feature.

4.2.9 CLI Commands

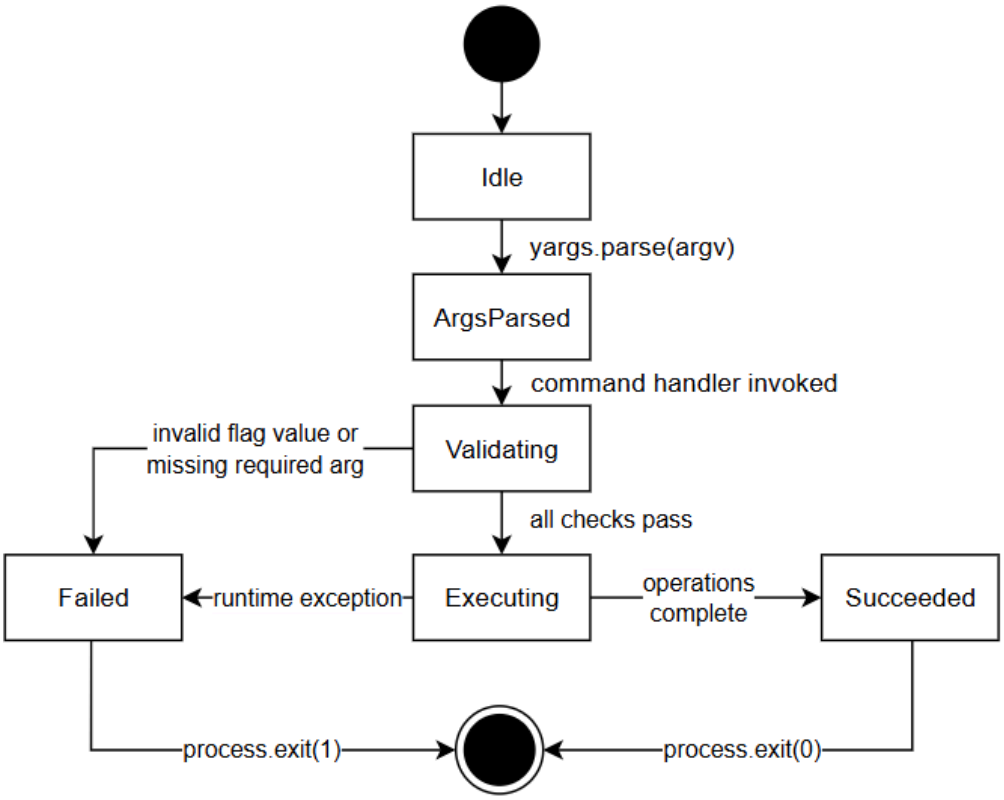

Figure 24: CLI Lifecycle State Diagram

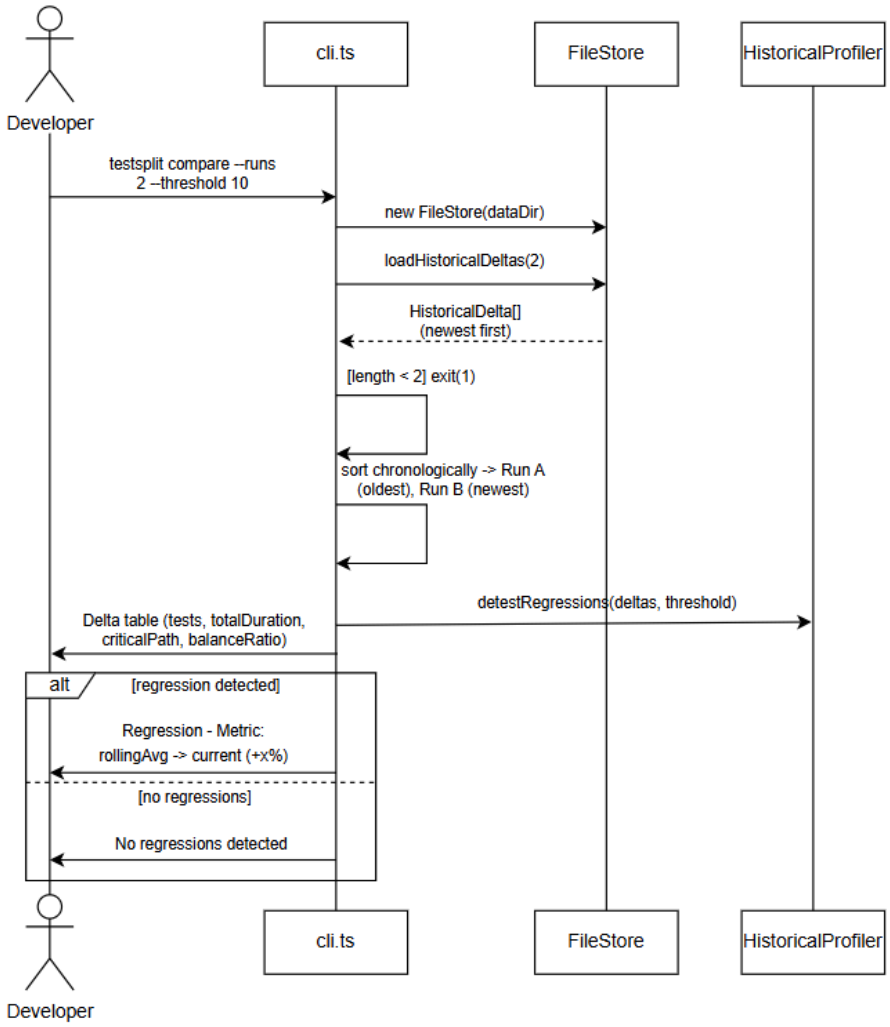

Figure 25: Compare Command State Diagram

The CLI is implemented using yargs and exposes eight commands, each registered with a dedicated handler in cli.ts. All commands follow a consistent pattern: validate inputs, resolve file paths, invoke the relevant engine or generator, and print structured output to the terminal using chalk for colour-coded feedback.

1. **profile**: accepts a JUnit XML path and job count, runs TestSplitEngine, and prints a profile summary with scheduling metrics and job distribution table.
2. **generate-config**: accepts a JUnit XML path, target platform, and job count. Auto-detects dependency constraints, lifecycle services, and container image before invoking the appropriate generator and writing the output YAML.
3. **generate-dockerfile**: accepts an optional --pom path and --out path, parses the Java version from pom.xml, and writes an eclipse-temurin Dockerfile to disk.

4. **compare**: loads two stored profiles from FileStore and prints a delta table showing duration changes and regression flags per test.
5. **benchmark**: runs the scheduler against a stored profile across a range of job counts and prints a comparison table of makespan and speedup estimates.
6. **validate**: parses an existing CI YAML file and runs it through CISchemaValidator and YAMLSyntaxValidator, reporting any structural or syntax issues.
7. **run**: invokes ParallelRunner with the current job distribution, executing test subsets in parallel across spawned child processes.
8. **dashboard**: builds the frontend if needed, starts the Fastify API on port 3001, and opens the browser.

The CLI command lifecycle state diagram [Figure 25] illustrates the state transitions across a full TestSplit workflow from invocation to output.

4.2.10 Frontend Test Suite

The frontend is tested using Vitest with a jsdom environment, providing a browser-like DOM for component rendering without a real browser. Tests are organised under src/frontend/__tests__/ in a structure that mirrors src/frontend/src/, with subdirectories for pages, components, hooks, and lib utilities.

React Testing Library is used to render components and assert on their output. Custom hooks are tested using renderHook. SWR cache isolation is enforced per test by wrapping each renderHook call in a SWRConfig provider backed by a fresh Map, preventing request deduplication from carrying state between tests. Heavy dependencies such as Recharts panel components and motion/react are mocked at the module level to keep tests fast and focused on page logic.

Code coverage is collected via @vitest/coverage-v8 and reported in lcov format. The frontend test suite runs as a dedicated frontend_tests job in the GitLab CI pipeline and is enforced locally via a Husky pre-push hook.

4.3 Algorithm Implementation

4.3.1 LPT Implementation

LPT is implemented in LPTScheduler.ts and follows a straightforward greedy assignment strategy. On invocation, SchedulerValidator first checks that the task list is non-empty and that jobCount is a positive integer that does not exceed the number of available tasks.

Tasks are sorted in descending order of smoothed duration. N empty job buckets are initialised, each with a totalTime of zero. The scheduler iterates over the sorted task list, assigning each task to the bucket with the current minimum totalTime, then incrementing that bucket's total by the task's duration. This guarantees the longest tasks are distributed first, reducing the likelihood of a single long test dominating one job while others finish early.

When a dependency map is present, a topological sort is applied to the task list before sorting by duration. Tasks with unsatisfied dependencies are deferred until their predecessors have been assigned, preserving dependency order while still applying the LPT heuristic within each dependency level.

On completion, SchedulingMetrics computes the makespan as the maximum totalTime across all buckets, the balance ratio as the ratio of maximum to minimum bucket load, the critical path length, and the estimated speedup over sequential execution. The result is returned as a JobDistribution containing the assigned jobs and computed metrics.

4.3.2 MULTIFIT Implementation

MULTIFIT is implemented in MULTIFITScheduler.ts using a binary search over candidate makespan values, with First Fit Decreasing (FFD) bin packing applied at each iteration to determine feasibility.

Tasks are sorted in descending order of duration, matching the FFD precondition. The binary search is initialised with a lower bound of the maximum single task duration (the minimum possible makespan) and an upper bound of the total duration of all tasks (the worst case where all tasks run sequentially). At each iteration, a midpoint capacity is selected, and FFD is applied. Tasks are assigned in order to the first job bucket with sufficient remaining capacity. If all tasks fit within N buckets at the candidate capacity, the upper bound is tightened. If not, the lower bound is raised. The search converges after a fixed number of iterations, returning the tightest feasible makespan found.

The FFD inner loop iterates over the sorted task list and, for each task, scans the existing buckets for the first bucket with remaining capacity. If no bucket can accommodate the task at the current candidate capacity, FFD reports infeasibility, and the binary search raises the lower bound.

As with LPT, SchedulerValidator validates inputs before execution, and SchedulingMetrics computes the makespan, balance ratio, and speedup on completion. The same JobDistribution interface is returned, making LPT and MULTIFIT fully interchangeable from the Configuration Generator's perspective.

5. Sample Code

5.1 LPT Scheduler Core Loop

The schedule() method in LPTScheduler.ts implements the core greedy assignment described in Section 2.2.1. It receives the full task list and a target job count, distributes tasks to jobs one at a time using a minimum-load selection rule, and returns a JobDistribution containing the assignment and post-scheduling metrics.

```
schedule(tasks: Task[], jobCount: number): JobDistribution {
  validateInputs(tasks, jobCount); // Explicitly validate to satisfy LPT assumptions

  const sorted = this.topologicalLptOrder(tasks);
  const jobs: Job[] = [];

  for (let i = 0; i < jobCount; i++) {
    jobs.push(new Job(i));
  }

  for (const task of sorted) {
    const lightestJob = jobs.reduce((min, job) => {
      if (job.totalTime < min.totalTime) return job;
      if (job.totalTime === min.totalTime && job.tasks.length < min.tasks.length) return job; //
tie-break by task count to spread zero-duration tests evenly
      return min;
    });
    lightestJob.addTask(task);
  }

  validateOutput(tasks, jobs);

  const totalDuration = tasks.reduce((sum, t) => sum + t.duration, 0);

  return {
    jobs,
    jobCount,
    totalDuration,
    metrics: computeMetrics(jobs, totalDuration)
  };
}
```

Listing 1: LPT Scheduler Core Assignment (schedule())

- **validateInputs** enforces the preconditions LPT requires: a non-empty task list, a positive integer job count, and numeric durations on every task. Keeping validation separate from the scheduling logic ensures the algorithm's assumptions are explicitly satisfied rather than silently depended upon, consistent with the independence requirement discussed in Section 2.2.4.
- **topologicalLptOrder** replaces a plain descending sort when test dependencies are present. The full implementation is covered in Section 5.2. When no dependencies exist in the task list, it reverts to a standard descending-duration sort, producing identical behaviour to the pure LPT case.

- **Job bucket initialisation** N empty Job buckets are created, each starting with totalTime = 0. The number of buckets matches the requested job count exactly.
- **The reduce call** selects the job with the smallest accumulated load at each step, the direct implementation of the LPT greedy rule. The tie-break on tasks.length handles zero-duration tests: when two jobs have equal accumulated time, the one with fewer tasks is preferred, spreading zero-duration tests evenly rather than allowing them to pile onto whichever job was selected first.
- **validateOutput** confirms that every input task appears in exactly one job after assignment. This partition-invariant check catches any edge case in which a task was silently dropped or duplicated during the greedy loop.
- **The returned JobDistribution** includes the job assignments, total suite duration, and metrics computed by computeMetrics: makespan, balance ratio, critical path, and estimated speed-up. This output feeds directly into the Configuration Generator and the dashboard.

5.2 MULTIFIT binary search with FFD

The schedule() method in MULTIFITScheduler.ts implements MULTIFIT [26], [27], an alternative to LPT that finds a near-optimal job distribution by running a binary search over candidate makespan values, calling First Fit Decreasing (FFD) bin packing at each iteration to test feasibility. It exposes the same interface as LPTScheduler and returns an identical JobDistribution, making the two algorithms fully interchangeable from the Configuration Generator's perspective.

```
 // 7 iterations gives a worst-case makespan ratio of <= 1.22 * OPT - (Coffman, Garey, Johnson 1978
see ScienceDirect reference above).
  private static readonly ITERATIONS = 7;

  schedule(tasks: Task[], jobCount: number): JobDistribution {
    validateInputs(tasks, jobCount);

    const sorted = [...tasks].sort((a, b) => b.duration - a.duration);
    const totalDuration = tasks.reduce((sum, t) => sum + t.duration, 0);
    const maxTask = sorted[0].duration;

    // Theoretical lower bound: makespan can never be less than the largest task or the average load per
job.
    let lo = Math.max(maxTask, totalDuration / jobCount);
    // Upper bound: all tasks sequentially in one job.
    let hi = totalDuration;

    let bestAssignment: Job[] | null = null;

    for (let iter = 0; iter < MULTIFITScheduler.ITERATIONS; iter++) {
      const mid = (lo + hi) / 2;
```

```
    const assignment = this.ffd(sorted, jobCount, mid);

    if (assignment !== null) {
      bestAssignment = assignment; // Feasible, record and try a tighter makespan.
      hi = mid;
    } else {
      lo = mid; // Infeasible, relax the makespan.
    }
  }

  /**
   * hi is always feasible after the first iteration (lo is provably feasible at start).
   * If binary search never found a feasible mid, fall back to the upper bound.
   */
  if (bestAssignment === null) {
    bestAssignment = this.ffd(sorted, jobCount, hi)!;
  }

  validateOutput(tasks, bestAssignment);

  return {
    jobs: bestAssignment,
    jobCount,
    totalDuration,
    metrics: computeMetrics(bestAssignment, totalDuration)
  };
}
```

Listing 2: MULTIFIT Binary Search (schedule())

- **ITERATIONS = 7** fixes the number of binary search iterations. Per Coffman, Garey and Johnson [26], seven iterations is sufficient to guarantee a worst-case makespan ratio of no more than 1.22 × OPT, meaning the result is always within 22% of the theoretical optimum regardless of input.
- **The lower bound lo** is initialised to Math.max(maxTask, totalDuration / jobCount) rather than zero. The longest single task is the theoretical minimum makespan, and the average load per job provides a second lower bound when tasks are more evenly distributed. Taking the maximum of both gives a tighter starting range and improves convergence stability, as documented in Section 7.2.
- **In the binary search loop,** at each iteration, the midpoint capacity, mid, is passed to ffd(), which attempts to assign all tasks to jobCount bins without exceeding that capacity. A non-null result means the capacity is feasible: bestAssignment is updated and hi is tightened. A null result means the capacity is too tight and lo is raised. After seven iterations, hi holds the tightest feasible makespan found. The ffd() implementation uses a 1e-9 epsilon tolerance on bin capacity

comparisons to prevent floating-point rounding errors from incorrectly rejecting feasible assignments, as described in Section 7.2.

- **The bestAssignment === null fallback** since lo is provably feasible at initialisation, this is a safety net rather than an expected code path, handling edge cases where the binary search never produced a feasible mid.
- **validateOutput** confirms the partition invariant after the binary search completes: every input task appears in exactly one job, consistent with the same check in LPTScheduler.
- **The returned JobDistribution** is identical in structure to LPT's output, containing job assignments, total suite duration, and metrics computed by computeMetrics: makespan, balance ratio, critical path, and estimated speed-up.

5.3 Exponential smoothing in HistoricalProfiler

computePerTestStats() in HistoricalProfiler.ts computes per-test statistics across all stored profiling runs and applies exponential smoothing to each test's duration, producing the weighted duration values used by the Scheduling Engine to distribute future jobs. It also flags tests as unstable or outliers based on their variance across runs, which is surfaced in the dashboard's Instability view.

```
private computePerTestStats(
  testDurationMap: Record<string, number[]>,
): Record<string, HistoricalTestStats> {
  const stats: Record<string, HistoricalTestStats> = {};

  for (const [testName, durations] of Object.entries(testDurationMap)) {
    const runCount = durations.length;
    const mean =
      runCount === 0
        ? 0
        : durations.reduce((sum, d) => sum + d, 0) / runCount;
    let smoothedMean = mean;

    if (runCount > 1) {
      const previousMean =
        durations.slice(0, -1).reduce((sum, d) => sum + d, 0) /
        (runCount - 1);

      smoothedMean =
        HistoricalProfiler.SMOOTHING_FACTOR * mean +
        (1 - HistoricalProfiler.SMOOTHING_FACTOR) * previousMean;
    }

    const variance = runCount === 0 ? 0 : durations.reduce((sum, d) => sum + Math.pow(d - mean, 2),
```

```
0) / runCount;
    const stdDev = Math.sqrt(variance);
    const min = runCount === 0 ? 0 : Math.min(...durations);
    const max = runCount === 0 ? 0 : Math.max(...durations);
    const coefficientOfVariation = mean > 0 ? stdDev / mean : 0;
    const unstable = coefficientOfVariation > HistoricalProfiler.INSTABILITY_THRESHOLD;
    const zeroDuration = durations.some((d) => d === 0);

    stats[testName] = {
      testName,
      runCount,
      meanDuration: smoothedMean,
      variance,
      stdDev,
      min,
      max,
      coefficientOfVariation,
      unstable,
      zeroDuration,
      isOutlier: false,
    };
  }

  const means = Object.values(stats).map((s) => s.meanDuration);
  if (means.length > 1) {
    const outlierThreshold = computeOutlierThreshold(means);

    for (const s of Object.values(stats)) {
      s.isOutlier = s.meanDuration > outlierThreshold;
    }
  }

  return stats;
}
```

Listing 3: Exponential Smoothing and Instability Detection (computePerTestStats())

- **mean computation**, the arithmetic mean is calculated across all recorded durations for each test. This serves as the current run's observed value fed into the smoothing formula.
- **The runCount > 1 guard**: smoothing is applied only when at least two runs exist. On the first run, smoothedMean equals the raw mean, meaning new tests enter the profile with their actual observed duration rather than a value pulled toward zero. From the second run onwards, the formula SMOOTHING_FACTOR * mean + (1 - SMOOTHING_FACTOR) * previousMean is

applied, where SMOOTHING_FACTOR is 0.6 [25]. This weights the current run at 60% and the historical average at 40%, keeping the profiler responsive to genuine performance changes while dampening noise from one-off slow tests.

- **coefficientOfVariation**, computed as stdDev / mean, is the normalised measure of variability used to determine instability [25]. A raw standard deviation cannot be compared meaningfully across tests with different mean durations; a 50ms deviation means something different for a 100ms test than for a 10,000ms test. The coefficient of variation normalises for this, making the instability threshold consistent across the entire suite.
- **unstable**, a test is flagged as unstable when its coefficient of variation exceeds INSTABILITY_THRESHOLD, set to 0.5. This means the standard deviation is more than half the mean, indicating that the test's duration is highly variable relative to its average and therefore unreliable as an input to the scheduler.
- **zeroDuration**, flags tests that recorded a duration of zero in any run. Zero-duration entries typically indicate a missing or unparseable time attribute in the Surefire XML report rather than a genuinely instant test, and are surfaced separately to alert the developer.
- **Outlier detection**, after per-test stats are computed, computeOutlierThreshold is called on the array of all smoothed means to identify tests whose duration is statistically anomalous relative to the rest of the suite. Tests exceeding the threshold have isOutlier set to true, highlighting them in the dashboard as candidates for investigation or manual splitting.

5.4 Topological Sort + LPT Hybrid

topologicalLptOrder() in LPTScheduler.ts resolves the conflict between LPT's requirement for a globally sorted task list and the constraint that dependent tests must execute in a fixed order. As documented in Section 7.1, a plain descending sort cannot be applied when dependencies are present because it would freely reorder tasks whose relative sequence is fixed by annotations such as JUnit 5's @Order or TestNG's dependsOnMethods. The solution is a hybrid of Kahn's topological sort algorithm [28] and the LPT heuristic, applying duration-based ordering within each dependency frontier rather than globally across all tasks.

```
private topologicalLptOrder(tasks: Task[]): Task[] {
  const taskById = new Map<string, Task>();
  const indegree = new Map<string, number>();
  const adjacency = new Map<string, string[]>();

  for (const task of tasks) {
    taskById.set(task.id, task);
    indegree.set(task.id, 0);
    adjacency.set(task.id, []);
  }

  for (const task of tasks) {
```

```
    const uniqueDependencies = new Set(task.dependencies ?? []);
    for (const dependencyId of uniqueDependencies) {
      adjacency.get(dependencyId)!.push(task.id);
      indegree.set(task.id, indegree.get(task.id)! + 1);
    }
  }

  const ready: Task[] = tasks.filter((task) => indegree.get(task.id) === 0);
  const ordered: Task[] = [];

  while (ready.length > 0) {
    ready.sort((a, b) => b.duration - a.duration || a.id.localeCompare(b.id));
    const current = ready.shift()!;
    ordered.push(current);

    for (const dependentId of adjacency.get(current.id)!) {
      const nextIndegree = indegree.get(dependentId)! - 1;
      indegree.set(dependentId, nextIndegree);
      if (nextIndegree === 0) {
        ready.push(taskById.get(dependentId)!);
      }
    }
  }

  if (ordered.length !== tasks.length) {
    throw new Error('Dependency cycle detected in tasks');
  }

  return ordered;
}
```

Listing 4: Topological Sort + LPT Hybrid (topologicalLptOrder())

- **taskById, indegree, and adjacency maps** are the three data structures implementing Kahn's algorithm scaffolding [28]. indegree tracks how many unresolved dependencies each task has. adjacency maps each task to the tasks that depend on it. taskById allows retrieving tasks by ID when adding newly unblocked tasks to the ready queue. All three are populated in a single pass over the task list before scheduling begins.
- **uniqueDependencies** wraps task.dependencies in a Set before iterating, preventing duplicate dependency edges from inflating a task's indegree when the same dependency appears multiple times in the annotation.

- **The ready queue** is initialised with all tasks whose indegree is zero, meaning they have no unresolved dependencies and are immediately eligible for scheduling. This is the starting frontier from which the hybrid algorithm proceeds.
- **ready.sort() inside the while loop** is the key insight of the hybrid. At each iteration, all currently unblocked tasks are sorted in descending order by duration, and the longest is selected. This applies the LPT heuristic within each dependency frontier rather than globally, prioritising the longest available task at every step while still respecting dependency order.
- **The secondary sort on a.id.localeCompare(b.id)** ensures deterministic output when two tasks have identical durations. Without it, tasks with equal durations could be assigned in different orders across runs on the same input, resulting in inconsistent CI configurations.
- **Decrementing indegree** after a task is scheduled, the indegree of every task that depends on it is decremented by one. Any task whose indegree reaches zero is added to the ready queue, making it eligible for scheduling at the next iteration. This is the standard Kahn's algorithm mechanism for progressively unlocking the dependency graph.
- **The cycle detection check** if ordered.length !== tasks.length after the loop completes, some tasks were never added to the ordered list because their indegree never reached zero, which can only happen if a cycle exists in the dependency graph. The method throws a descriptive error rather than returning a partial or silent result, aborting scheduling before an invalid distribution is produced.

5.5 JUnit XML parser

parseJUnitXMLFile() in JUnitXMLParser.ts parses a single Surefire XML report [14], [21] and extracts individual test results into a TestResult[] array consumed by the Profiling Engine. It handles the two structural variants produced by JUnit 4 and JUnit 5, normalises test names, class names, and durations into a consistent format, and gracefully degrades on malformed or incomplete input rather than throwing an exception.

```
function parseJUnitXMLFile(filePath: string): TestResult[] {
  const xml = readFileSync(filePath, 'utf-8');

  // Validate XML structure (warns but continues on error)
  validateXMLStructure(xml, filePath);

  const parser = new XMLParser({
    ignoreAttributes: false,
    attributeNamePrefix: '',
  });

  let parsed: any;
  try {
    parsed = parser.parse(xml);
```

```
} catch {
  console.warn(`[Parser] ${filePath}: Failed to parse XML`);
  return [];
}

let suites: any[];

if (parsed.testsuite) {
  suites = [parsed.testsuite];
} else if (parsed.testsuites?.testsuite) {
  suites = Array.isArray(parsed.testsuites.testsuite)
    ? parsed.testsuites.testsuite
    : [parsed.testsuites.testsuite];
} else {
  console.warn(`[Parser] ${filePath}: Missing <testsuite> or <testsuites> element`);
  return [];
}

const results: TestResult[] = [];

for (const suite of suites) {
  if (!suite.testcase) continue;

  // Startup/teardown durations extracted from suite properties if present
  const properties = suitePropertyMap(suite);

  const cases = Array.isArray(suite.testcase)
    ? suite.testcase
    : [suite.testcase];

  for (const tc of cases) {
    let status: 'passed' | 'failed' | 'skipped' = 'passed';
    if (tc.skipped !== undefined) status = 'skipped';
    else if (tc.failure !== undefined || tc.error !== undefined) status = 'failed';

    const testName = tc.classname && tc.name
      ? `${tc.classname}.${tc.name}`
      : (tc.name ?? 'unknown-test');

    const className = typeof tc.classname === 'string'
      ? tc.classname
      : typeof suite.name === 'string' ? suite.name : undefined;
```

```
    let duration = 0;
    if (tc.time !== undefined) {
      const parsedTime = Number(tc.time);
      if (!Number.isNaN(parsedTime)) duration = parsedTime;
    }

    results.push({
      name: testName,
      duration,
      status,
      classname: className,
      filePath: filePathFromClassName(className),
      packageName: packageFromClassName(className),
    });
   }
  }

  return results;
}
```

Listing 5: JUnit XML Parser (parseJUnitXMLFile())

- **validateXMLStructure** called before parsing begins, this performs a schema-level check using a DOM parser to verify the root element is <testsuite> or <testsuites> and that required attributes are present. Critically, it warns rather than throws on failure, meaning a structurally suspect file is still attempted rather than silently dropped.
- **The try/catch around parser.parse()** fast-xml-parser [22] throws on malformed XML. Catching here and returning an empty array ensures a single invalid file does not abort parsing of an entire directory of reports, consistent with the robustness requirement described in Section 4.2.1.
- **The testsuite vs testsuites branch** JUnit 4 Surefire reports use a single <testsuite> root element, while JUnit 5 reports wrap multiple suites inside a <testsuites> parent. The branch handles both, normalising either structure into a flat suites array before the test case loop runs. The inner Array.isArray guard on parsed.testsuites.testsuite handles the further edge case where a <testsuites> root contains only one <testsuite>, which fast-xml-parser returns as an object rather than an array.
- **The Array.isArray guard on suite.testcase** the same single/array quirk applies at the test case level. When a suite contains exactly one test case, fast-xml-parser produces a plain object rather than a single-element array. Wrapping it in an array ensures the loop handles both cases without branching further downstream.

- **Status mapping** skipped is checked before failure and error because a skipped test in Surefire XML can carry both a skipped element and an empty failure element, depending on the framework version. Checking skipped first ensures it is not incorrectly reported as failed.
- **className fallback to suite.name** when a test case does not carry its own classname attribute, the enclosing suite's name is used instead. This handles older JUnit 4 report formats where the class name is recorded at the suite level rather than per test case.
- **Duration extraction** tc.time is coerced to a number and checked with isNaN before assignment. A missing or unparseable time attribute defaults to 0 rather than propagating NaN into the profiling pipeline, consistent with the edge-case handling described in Section 4.2.1.

6. Results & Evaluation

6.1 Experimental Setup

To evaluate the effectiveness of TestSplit, validation was conducted against apache/commons-lang, an open-source Java library maintained by the Apache Software Foundation. The repository contains 231 test classes and 64,725 individual test cases.

6.1.1 Baseline Environment

Four baselines were established for each validation repository:

- **Local sequential execution:** mvn test executed directly on a developer machine (Apple M-series, macOS, Java 25, cold Maven cache).
- **Original CI pipeline:** The repository's unmodified GitHub Actions workflow, run as-is.
- **Controlled sequential CI:** A single GitHub Actions job (ubuntu-latest, Java 17, cold cache) running the full test suite sequentially.
- **TestSplit pipeline:** The TestSplit CLI was used to profile surefire XML reports, apply the LPT scheduling algorithm, and generate a parallelised GitHub Actions configuration. The generated pipeline was executed on a fork of each repository under identical conditions to the controlled baseline (ubuntu-latest, Java 17, cold cache)

All CI runs were performed on a fork of each repository to avoid interfering with upstream workflows. Caches were cleared between baseline and TestSplit runs to ensure cold-cache comparisons.

6.1.2 Tools & Versions

Component	Version
TestSplit	0.1.0
GitHub Actions Runner	ubuntu-latest

Java	Temurin 17
Maven	3.9.14 (latest version to date)
Scheduling Algorithm	Longest Processing Time

Table 3: Tools & Version for Validation

6.2 Performance Results

This section presents the performance results obtained from running TestSplit against each validation repository. For each repository, three sequential baselines are compared against the TestSplit-generated parallel pipeline. All results were recorded under the conditions described in Section 6.1.

6.2.1 apache/commons-lang

apache/commons-lang is an open-source Java utility library maintained by the Apache Software Foundation, providing helpers for string manipulation, concurrency, reflection, and date/time utilities.

Metric	**Value**
Test Classes	231
Total Tests	64,725
Parallel Jobs	14
Scheduling Algorithm	LPT

Table 4: Metric Summary for apache/commons-lang Project

Sequential Baseline	
Scenario	**Time (minutes)**
Original CI matrix (Slowest job)	17:10
Maven Test (Local machine test)	13:29
Controlled sequential CI (No Java matrix)	5:27
TestSplit pipeline	4:20

Table 5: Results from Baseline Scenarios

Comparison	Before	After	Speedup
Vs Original CI Matrix	17:10	4:20	3.9x
Vs Maven Test (Local machine test)	13:29	4:20	3.1x
Vs Controlled (No matrix) Sequential CI	5:27	4:20	1.26x
Test Execution Only (Excluding build overhead)	5:27	2:26	2.24x

Table 6: Comparison Table for apache/commons-lang Project

The profiler identified 55,665 of 64,725 tests (86%) as reporting zero duration in the surefire XML output sub-millisecond tests that cannot be individually timed. These were distributed with equal weight across jobs, limiting scheduling accuracy. Despite this, the LPT algorithm correctly identified the critical path bottleneck (ExtendedMessageFormatTest, 78s) and isolated it to Job 1. The balance ratio was 2.35, with the slowest job (2:26) taking approximately 4x longer than the fastest (36s).

The modest 1.26x controlled CI speedup is primarily attributable to two factors: fixed build overhead (1:48, representing 41% of total pipeline duration) and limited timing precision in the surefire XML reports. The test execution phase alone achieved a 2.24x speedup, and the pipeline achieved a 3.1x improvement over local sequential execution, the most representative measure of real-world developer feedback time.

6.2.2 jsoup

jhy/jsoup is a Java HTML parser library with 1,853 tests across 62 test classes.

Running TestSplit against the jsoup surefire reports produced the following profile:

- **Critical path:** 5.05s (dominated by a single network integration test: ProxyTest.canAuthenticateToProxy)
- **Balance ratio:** 9.14
- **1,297** of **1,853** tests (70%) reported zero duration (sub-millisecond)

The high balance ratio and near-zero median test duration indicated the suite was dominated by one slow outlier, making it a poor candidate for fine-grained parallelisation. TestSplit's adaptive job count cap detected that the total profiled duration of 7.73s was too short to justify multiple jobs, and reduced the target to 2, reasoning that runner startup overhead would otherwise dominate.

Results		
Configuration	Jobs	Overall Pipeline Time
Original build.yml	1	2m 14s
TestSplit	2	1m 17s

Table 7: Jsoup Result Comparison vs TestSplit

Splitting into two jobs still achieved a 1.74x reduction in job time. Even for imbalanced suites, coarse-grained parallelism, where the slower job runs the long outlier test, and the faster job handles the remaining 61 classes, can deliver meaningful improvement without generating excessive runner overhead.

6.2.3 bonigarcia/mastering-junit5

bonigarcia/mastering-junit5 is a multi-module Maven project containing 43 modules covering JUnit 5 features including unit, integration, and end-to-end testing patterns. Unlike the previous two repositories, this was selected to validate TestSplit against automated system tests using Selenium WebDriver, exercised through the junit5-cucumber-selenium module which runs Cucumber scenarios against a real browser via Xvfb.

TestSplit profiled the Cucumber-Selenium test suite, generated a parallel GitHub Actions workflow, and the pipeline was pushed to a fork of the repository. Both the build job and test job completed successfully in CI. All Selenium scenarios passed without modification, confirming that TestSplit's config generation correctly preserves browser-dependent setup steps (Xvfb display configuration) and does not break automated system tests.

6.3 Limitations

6.3.1 Maven Surefire Dependency

TestSplit parses JUnit XML reports in the format produced by Maven Surefire. Projects using Gradle, Bazel, or other build systems do not produce Surefire-compatible output and cannot be profiled without manual conversion. Similarly, JavaScript and TypeScript projects using Jest produce a JSON report format that is not currently supported. The tool is therefore limited to Maven-based Java projects in its current form.

6.3.2 Single Module-Maven Projects

The CI generator assumes a single-module Maven project layout in which all compiled classes and test sources reside under a single target/ directory. Multi-module Maven projects require additional flags (-pl, install steps for SNAPSHOT dependencies) that the generator does not produce automatically. This was encountered during validation against google/guava, which uses a guava-tests/ submodule structure and

would require manual patches to the generated configuration, hence the exclusion of the project during validation.

6.3.3 Annotation Processor Dependencies

Some Java projects rely on annotation processors that generate source files at compile time and are required by the test suite at runtime. When tests are split across parallel Maven jobs, each job invokes Maven independently. If an annotation processor produces output not included in the uploaded build artifact, individual test jobs fail to compile. This was encountered during validation against google/truth (hence its exclusion on results during validation), where generation caused each split job to fail. This is a fundamental constraint of the phased build-then-test pipeline design and cannot be resolved without deeper knowledge of each project's build graph.

6.3.4 Scheduling Based on Historical Duration Estimates

The LPT scheduler distributes tests based on smoothed duration estimates from previous runs. If the test suite changes significantly between runs, new tests added, existing tests refactored, or timing characteristics altered by dependency updates, the historical profile becomes stale and the generated distribution may be poorly balanced until enough new runs have been collected for the smoothed estimates to converge. A project's first run has no historical data and falls back to a single-run profile, which may contain outliers or timing noise.

6.3.5 GitHub Actions and GitLab CI Only

The configuration generator targets GitHub Actions and GitLab CI. Projects using Jenkins, CircleCI, Bitrise, or other CI platforms cannot use the generated output directly. The phased pipeline pattern (build job followed by N parallel test jobs with artifact sharing) does not translate directly to other pipeline syntaxes without reimplementing the generator for each platform.

6.4 Functional Requirements Evaluation

6.4.1 FR1 - JUnit XML Parsing

Status: Fully met

JUnitXMLParser in src/backend/parser/JUnitXMLParser.ts handles both flat <testsuite> and nested <testsuites> root elements, covering JUnit 4 and JUnit 5 Surefire report structures. Both single file and directory inputs are accepted via parseJUnitXMLPath(), which recurses into subdirectories and filters for *.xml files. @xmldom/xmldom validates the XML structure before parsing, and malformed files emit a warning rather than crashing the process. Each <testcase> element yields a TestResult carrying name, className, duration, status, filePath, and packageName. Suite-level startup and teardown durations are extracted from <properties> blocks where present.

The parser was exercised against real-world Surefire reports from apache/commons-lang (456 XML files, 52,000+ test cases) and jhy/jsoup (1,308 test cases).

6.4.2 FR2 - Test Profiling Engine

Status: Fully met

Profiler in src/backend/profiler/core/Profiler.ts computes per-test statistics including mean duration, variance, standard deviation, min, max, coefficient of variation, and outlier and zero-duration flags. HistoricalProfiler extends this with exponential smoothing at α = 0.6, weighting each new observation as 0.6 * current + 0.4 * previous. Tests are flagged as unstable when their coefficient of variation exceeds a threshold of 0.5. EnvironmentCollector captures CPU model, core count, OS version, Node.js version, available memory, and Docker environment at profiling time.

The --risk-factor parameter (k) biases scheduling weights to meanDuration + k * stdDev, making the profiler defensively conservative when test timing is non-uniform. Outlier detection was observed during validation: two jsoup tests and 21 commons-lang tests were flagged as outliers within single profiling runs.

6.4.3 FR3 - LPT Scheduling

Status: Fully met

LPTScheduler in src/backend/algorithm/core/LPTScheduler.ts sorts tasks by descending duration and assigns each to the job bucket with the minimum accumulated load. When a dependency map is present, a topological sort is applied before the LPT pass, preserving ordering constraints while applying the heuristic within each dependency level. MULTIFITScheduler provides an alternative via binary search over candidate makespan values with First Fit Decreasing bin packing at each iteration; both algorithms implement the same schedule(tasks, jobCount): JobDistribution interface and are fully interchangeable.

SchedulingMetrics computes makespan, balance ratio, ideal time, critical path, and estimated speedup over sequential execution. The computeAdaptiveJobCount utility in src/backend/utils/adaptiveJobCap.ts caps the job count when total suite duration is too short to justify the requested parallelism (MIN_VIABLE_JOB_S = 60 seconds per job), and is applied consistently in both the profile and generate-config commands.

6.4.4 FR4 - CI Configuration Generation

Status: Fully met

GitHubActionsGenerator produces a phased workflow with a build job that compiles the project and uploads build artifacts, followed by N parallel test jobs that each download the artifact and run mvn test -Dtest=ClassName1,ClassName2,..., GitLabCIGenerator follows the same pattern using GitLab CI YAML syntax. Both generators pass output through YAMLSyntaxValidator and CISchemaValidator before writing to disk. YAML.stringify is called with lineWidth: 0 to prevent line folding from inserting whitespace inside -Dtest= values.

Class-level task aggregation in TestSplitEngine groups per-method results by classname before scheduling, aligning task granularity with Maven Surefire's -Dtest=ClassName resolution and preventing memory exhaustion on large suites. The generator was validated against two repositories: apache/commons-lang (8 jobs, 3.9x speedup) and jhy/jsoup (2 jobs, 1.74x speedup).

6.4.5 FR5 - Local Data Storage

Status: Fully met

FileStore in src/backend/storage/FileStore.ts manages all persistent data under .data/ in the project directory. Profiles and distributions are stored as plain JSON. The historical profile is written as gzip-compressed JSON via zlib.gzipSync(). Historical deltas are stored as plain JSON up to a maximum of 50 uncompressed files, after which older entries are rotated to .json.gz and the originals are removed. Read operations transparently decompress .gz files with zlib.gunzipSync(). StoragePaths centralises all file path resolution, and SchemaVersions wraps stored distributions to support forward-compatible migrations.

6.4.6 FR6 - CLI Commands

Status: Fully met

The CLI in src/backend/cli/cli.ts registers eight commands via yargs: profile, generate-config, compare, benchmark, validate, run, dockerfile, and dashboard. All commands follow a consistent pattern: validate inputs, resolve paths, invoke the relevant engine or generator, and print structured output with chalk colour-coding. The profile and generate-config commands both apply the adaptive job count cap through the shared computeAdaptiveJobCount utility, ensuring the job count reported during profiling is consistent with the count used in the generated CI configuration.

6.4.7 FR7 - Local React Dashboard

Status: Fully met

The React dashboard is implemented across four pages in src/frontend/src/pages/: Overview, Durations, Scheduling, and Instability. The Fastify API in src/backend/api/server.ts exposes five endpoints: /api/health, /api/summary, /api/tests, /api/jobs, and /api/trends. The dashboard CLI command builds the frontend if no production build exists under src/frontend/dist/, starts the API server on port 3001, and opens the browser. The Overview page displays animated summary cards and trend charts; the Scheduling page renders the job distribution bar chart and balance metrics; the Instability page renders a coefficient of variation scatter plot to identify unstable and outlier tests.

The frontend is tested using Vitest and React Testing Library. Unit and component tests cover all four pages, shared components, custom hooks, and utility functions. Tests are colocated under src/frontend/__tests__/ and run as part of the CI pipeline via a dedicated frontend_tests job. Code coverage is collected using @vitest/coverage-v8.

6.4.8 FR8 - Containerised Execution

Status: Fully met

DockerfileGenerator in src/backend/generator/DockerfileGenerator.ts produces an eclipse-temurin:{javaVersion}-jdk-based Dockerfile with Maven dependency caching via dependency:go-offline. The Java version is extracted from pom.xml via PomParser, and the Maven wrapper (mvnw) is used when detected. The dockerfile CLI command writes the generated file to disk.

Container image injection is implemented in both generators: when a Dockerfile is present in the project root, the base image is injected into CI jobs via the container: field for GitHub Actions and the image: field for GitLab CI, ensuring test jobs execute in the same environment as local development.

6.4.9 FR9 - Historical Trend Analysis

Status: Fully met

HistoricalProfiler in src/backend/profiler/core/HistoricalProfiler.ts accumulates profiles across runs and applies exponential smoothing to maintain per-test smoothed duration estimates. generateHistoricalProfile() produces aggregate statistics including run count, average test duration, and per-test smoothed stats. The static detectRegressions() method compares the latest run against a rolling average of previous runs, flagging regressions when the relative change in critical path or balance ratio exceeds a configurable threshold (default 10%).

The compare command loads historical deltas from FileStore, prints a chronological table of run metrics with directional indicators (up arrow/down arrow/right arrow) and colour-coded changes, and invokes detectRegressions() to surface any metrics that have deteriorated beyond the threshold.

7. Problems Encountered

7.1 Topological Sort + LPT Hybrid

7.1.1 Problem

The Longest Processing Time (LPT) algorithm requires all tasks to be sorted by duration before scheduling. However, when test dependencies are present, tasks cannot be freely reordered and must respect dependency constraints. Kahn’s topological sort releases tasks incrementally as dependencies are resolved, which conflicts with LPT’s requirement for a globally sorted task list.

7.1.2 Solution

A hybrid approach was implemented using a ready queue within the topological sort process. At each iteration, all dependency-free tasks are added to the ready queue and sorted in descending order of

duration. The LPT heuristic is then applied within this subset, assigning the longest available task first. This preserves dependency correctness while maintaining LPT's load-balancing behaviour. Cycle detection is performed by verifying that all tasks are processed; if the final ordered list is incomplete, scheduling is aborted due to a detected dependency cycle.

7.2 MULTIFIT Binary Search with Floating-Point Precision

7.2.1 Problem

The MULTIFIT algorithm relies on a binary search over possible makespan values combined with First Fit Decreasing (FFD) bin packing. Floating-point precision errors can cause incorrect feasibility checks, where tasks that should fit within a bin are incorrectly rejected due to rounding inaccuracies.

7.2.2 Solution

An epsilon tolerance (1e-9) was introduced when comparing task durations against remaining bin capacity, ensuring that minor floating-point discrepancies do not affect feasibility decisions. Additionally, the binary search bounds were initialised using max(maxTaskDuration, totalDuration/jobCount) as the lower bound, rather than zero. This provides a mathematically valid starting range, improving convergence stability and ensuring the search space is well-defined.

7.3 Java Source Parsing Without an AST

7.3.1 Problem

Extracting test ordering and dependency information from Java source files without using a full Abstract Syntax Tree (AST) parser is challenging due to variations in formatting, whitespace, and language constructs. Naive regex approaches risk incorrectly matching keywords or missing valid method definitions.

7.3.2 Solution

A multi-layered regex strategy was implemented. Method extraction patterns were designed to handle multiline definitions and varying whitespace, while explicitly filtering out Java control-flow keywords to prevent false positives. Class name resolution follows a three-stage fallback approach:

1. Extract from package declarations
2. Infer from Maven directory structure
3. Default to unqualified class names

This approach achieves reliable parsing without the overhead of integrating a full AST parser.

7.4 Work Stealing Load Balancing

7.4.1 Problem

LPT scheduling relies on historical execution times, which may not accurately reflect real runtime behaviour. This can result in some workers finishing early while others remain overloaded, reducing overall parallel efficiency.

7.4.2 Solution

A work-stealing mechanism was introduced in the Parallel Runner. Each worker first executes its assigned tasks, then retrieves additional tasks from a shared queue if idle. If no shared tasks are available, the worker steals tasks from the busiest worker (i.e., the one with the largest remaining workload). Prioritising the heaviest worker ensures that the imbalance is progressively reduced, improving overall resource utilisation and reducing total execution time.

7.5 CLI Auto-Detection Chain

7.5.1 Problem

The generate-config command requires multiple pieces of contextual information (Dockerfile presence, Java version, dependencies, lifecycle services), which may or may not exist in a given project. Requiring users to manually specify all inputs would reduce usability, but automatic detection introduces complexity due to optional and interdependent steps.

7.5.2 Solution

A sequential auto-detection chain was implemented, where each detection stage (Dockerfile, pom.xml, source scanning, suite XML, lifecycle services) executes independently and passes its result forward. Each stage gracefully falls back to undefined when relevant inputs are absent, ensuring downstream components always receive valid inputs.

8. Future Work

TestSplit is a functional first version of an automated CI test-splitting tool. This section outlines areas identified for future development based on limitations and natural extensions observed during implementation.

8.1 Shared Profiling Backend

FileStore provides local-first storage suitable for individual developers. A PostgreSQL backend would enable team-shared profiling data, allowing multiple developers on the same project to contribute to and benefit from a shared historical profile.

8.2 Broader CI Platform Support

The current generators target GitHub Actions and GitLab CI. Support for CircleCI, Bitrise, and Jenkins pipelines would extend TestSplit's applicability to a wider range of development teams.

8.3 Maven Multi-Module Support

TestSplit currently operates on single-module Maven projects. Multi-module projects with nested pom.xml files and cross-module test dependencies are not handled and represent a meaningful extension for larger Java codebases.

8.4 ML-Based Scheduling

LPT and MULTIFIT are heuristic algorithms with known worst-case bounds. A machine learning approach trained on historical execution patterns could produce more accurate task-duration predictions and more balanced job distributions over time, particularly for test suites with high variability.

9. Conclusion

TestSplit was designed and implemented as a local, open-source tool to optimise test distribution in CI pipelines. Starting from JUnit XML output produced by Maven Surefire, the system profiles test execution times, applies the Longest Processing Time algorithm to distribute tests across parallel jobs, and generates ready-to-use YAML configuration files for GitHub Actions and GitLab CI, requiring no modifications to existing tests, build scripts, or repository structure.

The four objectives set out in Section 1.2 were each addressed by the implementation. CI pipeline runtime reduction is achieved through LPT and MULTIFIT scheduling, both of which produce near-optimal job distributions with measurable speedups over sequential execution. Workload balance is quantified through post-scheduling metrics, including makespan, balance ratio, and critical path, exposed via the CLI and dashboard. Minimal workflow disruption is maintained by generating additive configuration files that sit alongside existing pipelines without modifying them. Historical performance insight is provided through exponential smoothing across profiling runs, instability and regression detection, and a local React dashboard visualising trends over time.

Several non-trivial implementation challenges were encountered and resolved during development, including the topological sort and LPT hybrid for dependency-aware scheduling, floating-point precision handling in MULTIFIT's binary search, Java source parsing without an AST, work-stealing load balancing in the parallel runner, and the CLI auto-detection chain. Each is documented in Section 7.

TestSplit's current scope is intentionally constrained: it targets single-module Maven projects, consumes JUnit XML exclusively, and stores data locally. These boundaries reflect the priorities of the initial implementation rather than the approach's fundamental limitations. The extensions identified in Section 9, including shared profiling storage, broader CI platform support, multi-module Maven handling, and ML-based scheduling, represent natural next steps for a future version of the tool.

10. References

[1] W. Felidré, L. Furtado, D. da Costa, B. Cartaxo, and G. Pinto, “Continuous Integration Theater,” Jul. 02, 2019, *arXiv*: arXiv:1907.01602. doi: 10.48550/arXiv.1907.01602.

[2] O. Martin, “Unlocking the true potential of CI/CD pipelines for improved productivity,” Calibo. Accessed: Mar. 31, 2026. [Online]. Available: https://calibo.com/blog/ci-cd-pipelines-for-improved-productivity/

[3] “Four of the biggest challenges teams face with CI/CD | Stride.” Accessed: Mar. 31, 2026. [Online]. Available: https://www.stride.build/blog/four-of-the-biggest-challenges-teams-face-with-ci-cd

[4] R. L. Graham, “Bounds on Multiprocessing Timing Anomalies,” *SIAM J Appl Math*, vol. 17, no. 2, pp. 416–429, Mar. 1969, doi: 10.1137/0117039.

[5] X. Xiao, “A Direct Proof of the 4/3 Bound of LPT Scheduling Rule,” Jan. 2017. doi: 10.2991/fmsmt-17.2017.102.

[6] F. Della Croce and R. Scatamacchia, “The Longest Processing Time rule for identical parallel machines revisited,” *J. Sched.*, vol. 23, no. 2, pp. 163–176, Apr. 2020, doi: 10.1007/s10951-018-0597-6.

[7] “Choosing the runner for a job,” GitHub Docs. Accessed: Mar. 31, 2026. [Online]. Available: https://docs-internal.github.com/en/actions/how-tos/write-workflows/choose-where-workflows-run/choose-the-runner-for-a-job

[8] “CI/CD pipelines | GitLab Docs.” Accessed: Mar. 31, 2026. [Online]. Available: https://docs.gitlab.com/ci/pipelines/

[9] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” in *Proceedings of the April 18-20, 1967, spring joint computer conference*, in AFIPS ’67 (Spring). New York, NY, USA: Association for Computing Machinery, Apr. 1967, pp. 483–485. doi: 10.1145/1465482.1465560.

[10] M. Herlihy and N. Shavit, *The art of multiprocessor programming*, Nachdr. Amsterdam Heidelberg: Elsevier, 2009.

[11] “Scheduling: Theory, Algorithms, and Systems | Springer Nature Link.” Accessed: Mar. 31, 2026. [Online]. Available: https://link.springer.com/book/10.1007/978-3-031-05921-6

[12] M. Dell’Amico, “Scheduling With Inexact Job Sizes: The Merits of Shortest Processing Time First,” Jul. 10, 2019, *arXiv*: arXiv:1907.04824. doi: 10.48550/arXiv.1907.04824.

[13] S. Oaks, *Java performance: the definitive guide*. Sebastopol, CA: O’Reilly Media, 2014.

[14] “Introduction – Maven Surefire Plugin.” Accessed: Mar. 31, 2026. [Online]. Available: https://maven.apache.org/surefire/maven-surefire-plugin/

[15] “Continuous Integration: Improving Software Quality and Reducing Risk (The Addison-Wesley Signature Series) | Guide books | ACM Digital Library,” Guide books. Accessed: Mar. 31, 2026. [Online]. Available: https://dl.acm.org/doi/book/10.5555/1208841

[16] “Overview | Knapsack Pro Docs.” Accessed: Mar. 31, 2026. [Online]. Available: https://docs.knapsackpro.com/overview/

[17] “Find, Quarantine, and Fix Flaky Tests Instantly.” Accessed: Mar. 31, 2026. [Online]. Available: https://buildpulse.io/products/flaky-tests

[18] *Software Architecture in Practice, 4th Edition*. Accessed: Mar. 31, 2026. [Online]. Available: https://learning.oreilly.com/library/view/software-architecture-in/9780136885979/

[19] *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Accessed: Apr. 01, 2026. [Online]. Available: https://learning.oreilly.com/library/view/clean-architecture-a/9780134494272/

[20] *Design Patterns: Elements of Reusable Object-Oriented Software*. Accessed: Apr. 01, 2026. [Online]. Available: https://learning.oreilly.com/library/view/design-patterns-elements/0201633612/

[21] “maven.apache.org/surefire/maven-surefire-plugin/xsd/surefire-test-report.xsd.” Accessed: Apr. 01, 2026. [Online]. Available: https://maven.apache.org/surefire/maven-surefire-plugin/xsd/surefire-test-report.xsd
[22] “Overview :: JUnit User Guide.” Accessed: Apr. 01, 2026. [Online]. Available: https://docs.junit.org/6.0.3/overview.html
[23] “systeminformation.” Accessed: Apr. 01, 2026. [Online]. Available: https://systeminformation.io/gettingstarted.html
[24] R. J. Hyndman, “Forecasting: Principles & Practice”.
[25] “lpt4:3.pdf.” Accessed: Apr. 01, 2026. [Online]. Available: https://community.wvu.edu/~krsubramani/courses/sp14/approx/auxslinotes/lpt4:3.pdf
[26] “Multifit algorithm,” *Wikipedia*. Oct. 07, 2025. Accessed: Apr. 01, 2026. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Multifit_algorithm&oldid=1315555719#The_algorithm
[27] *Graph Algorithms the Fun Way*. Accessed: Apr. 01, 2026. [Online]. Available: https://learning.oreilly.com/library/view/graph-algorithms-the/9781098182519/
[28] “Workflow syntax for GitHub Actions,” GitHub Docs. Accessed: Apr. 01, 2026. [Online]. Available: https://docs-internal.github.com/en/actions/reference/workflows-and-actions/workflow-syntax
[29] “CI/CD YAML syntax reference | GitLab Docs.” Accessed: Apr. 01, 2026. [Online]. Available: https://docs.gitlab.com/ci/yaml/#image
[30] “Services | GitLab Docs.” Accessed: Apr. 01, 2026. [Online]. Available: https://docs.gitlab.com/ci/services/
[31] “Control startup order,” Docker Documentation. Accessed: Apr. 01, 2026. [Online]. Available: https://docs.docker.com/compose/how-tos/startup-order/
[32] *Designing Data-Intensive Applications, 2nd Edition*. Accessed: Apr. 01, 2026. [Online]. Available: https://learning.oreilly.com/library/view/designing-data-intensive-applications/9781098119058/
[33] “Introduction,” Fastify. Accessed: Apr. 01, 2026. [Online]. Available: https://fastify.dev/
[34] D. A. Norman, *The design of everyday things*, Revised and Expanded edition. New York, New York: Basic Books, 2013.
[35] “JUnit – About.” Accessed: Apr. 01, 2026. [Online]. Available: https://junit.org/junit4/
[36] “TestNG Documentation.” Accessed: Apr. 01, 2026. [Online]. Available: https://testng.org/
[37] “Postgres Module - Testcontainers for Java.” Accessed: Apr. 01, 2026. [Online]. Available: https://java.testcontainers.org/modules/databases/postgres/
[38] “MySQL Module - Testcontainers for Java.” Accessed: Apr. 01, 2026. [Online]. Available: https://java.testcontainers.org/modules/databases/mysql/
[39] “Kafka Module - Testcontainers for Java.” Accessed: Apr. 01, 2026. [Online]. Available: https://java.testcontainers.org/modules/kafka/
[40] “Testcontainers MongoDB Module,” Testcontainers. Accessed: Apr. 01, 2026. [Online]. Available: https://testcontainers.com/modules/mongodb/
[41] “Testcontainers Redis Module,” Testcontainers. Accessed: Apr. 01, 2026. [Online]. Available: https://testcontainers.com/modules/redis/
[42] “Testing Applications :: Spring Kafka.” Accessed: Apr. 01, 2026. [Online]. Available: https://docs.spring.io/spring-kafka/reference/testing.html#embedded-kafka-annotation
[43] “Testing Spring Boot Applications :: Spring Boot.” Accessed: Apr. 01, 2026. [Online]. Available: https://docs.spring.io/spring-boot/reference/testing/spring-boot-applications.html#testing.spring-boot-applications.autoconfigured-jpa
[44] “AutoConfigureTestDatabase (Spring Boot 4.0.5 API).” Accessed: Apr. 01, 2026. [Online]. Available: https://docs.spring.io/spring-boot/api/java/org/springframework/boot/jdbc/test/autoconfigure/AutoConfigureTestDatabase.html

[45] “Docker Compose,” Docker Documentation. Accessed: Apr. 01, 2026. [Online]. Available: https://docs.docker.com/compose/
[46] “Child process | Node.js v25.8.2 Documentation.” Accessed: Apr. 01, 2026. [Online]. Available: https://nodejs.org/api/child_process.html
[47] R. D. Blumofe and C. E. Leiserson, “Scheduling multithreaded computations by work stealing,” *J ACM*, vol. 46, no. 5, pp. 720–748, Sep. 1999, doi: 10.1145/324133.324234.
[48] “taskset(1) - Linux manual page.” Accessed: Apr. 01, 2026. [Online]. Available: https://man7.org/linux/man-pages/man1/taskset.1.html
[49] “eclipse-temurin - Official Image | Docker Hub.” Accessed: Apr. 01, 2026. [Online]. Available: https://hub.docker.com/_/eclipse-temurin
[50] “dependency:go-offline – Apache Maven Dependency Plugin.” Accessed: Apr. 01, 2026. [Online]. Available: https://maven.apache.org/plugins/maven-dependency-plugin/go-offline-mojo.html
[51] “Cache,” Docker Documentation. Accessed: Apr. 01, 2026. [Online]. Available: https://docs.docker.com/build/cache/